MosaicOpt: An Algebraic Foundation for Agentic Memory Databases

Anonymous Author(s)
Anonymous Institution
Anonymous City, Anonymous State, Anonymous Country
anonymous@example.com

Abstract

Production LLM agents accumulate multi-modal memory across two timescales: long-term facts and preferences, and short-term working context. A memory unit typically carries text, one or more dense vector embeddings, structured metadata, and graph relations. Useful queries against this store are conjunctive expressions that mix all four modalities, for example, "find passages about a fault-tolerance bug from the last 30 days, semantically similar to a stack trace, attributed to a known subsystem, and connected by a derivation link to a prior incident." The dominant engineering response, *multi-path recall* with application-level score fusion, has no principled semantics, no equivalence relation over plans, and breaks down on multi-vector conjunctions. We propose **MosaicOpt**, an *algebra-first* design that extends relational algebra with (i) a *materialization* operator μ that binds computed values (vector distances, BM25 scores) as first-class attributes, (ii) first-class vector, text, and graph predicates, and (iii) a graph expansion operator Γ. The optimizer is a Volcano/Cascades-style rule-based layer driven by an *equational theory* of seven foundational identities (M1–M5, G1, G2) over the extended operators, supplemented by the assumed classical A1–A8 (including join commutativity and associativity; A1 and A4 apply naturally to vector predicates), the derived chains (M4+M1, M5+G1, A4-vector+M1), the classical applications (A7-symmetry, A8-regrouping), the definitional τ-Λ-topK rewrite, and the row-preserving σ-τ commutation, combined with a cost-based layer that explicitly accounts for approximation. We prove the identities, give counterexamples for the non-identities (N1–N5) that classical intuition suggests but approximation, graph-traversal asymmetry, or the traversal-vs-origin column split break, and sketch a prototype and evaluation plan. The argument is foundational: agentic memory is a query problem, query problems benefit from algebraic foundations, and the right foundation is an extension of relational algebra.

Keywords

agentic memory, relational algebra, query optimization, vector search, hybrid retrieval, rule-based optimizer, equational theory

Running example: a real agentic-memory query

*"Find **fault-tolerance** passages from the **last 30 days**, **body**- and **summary**-similar to a stack trace, attributed to a known **subsystem**, linked by* `derived_from` *to a previous incident, **top 10** by (d_1, d_2)."*

$$
\begin{aligned}
Q_1 &= Mem \\
Q_2 &= \sigma_{tag=\text{ft},\ sub=\text{known},\ ts>\text{now}()-30d}(Q_1) \\
Q_3 &= \mu_{\|emb-stack\|\to d_1}\,\mu_{\|emb_alt-focus\|\to d_2}(Q_2) \\
Q_4 &= \sigma_{d_1<\tau_1\ \wedge\ d_2<\tau_2}(Q_3) \\
Q_5 &= \Gamma_{derived_from,\,d=1}(Q_4) \\
Q_6 &= \Lambda_{10}^{(d_1,d_2)}(Q_5)
\end{aligned}
$$

Figure 1: Running example: the same agentic-memory query as a natural-language ask (left, keywords highlighted) and as an algebraic expression in the extended relational algebra (right). The query exercises two vector columns (*emb* for body, *emb_alt* for summary) plus a tag predicate, a time window, a categorical subsystem filter, a one-hop graph expansion along *derived_from*, and a top-K over the materialised distance columns. An explicit score-fusion policy (e.g. $0.7\,d_1 + 0.3\,d_2$) is also expressible via the τ operator and is not exercised in this example.

1 Introduction & Motivation

LLM agents accumulate state across two timescales: *long-term memory* (persistent facts, preferences, episodic traces) and *short-term memory* (working context of an in-flight session). A memory unit carries raw text, one or more dense vector embeddings, structured metadata, and graph relations (`derived_from`, `cites`, etc.). Useful queries are *conjunctive* across all four modalities, "find passages about a fault-tolerance bug from the last 30 days, semantically similar to a stack trace, attributed to a known subsystem, and linked by `derived_from` to a previous incident" is representative.

The dominant engineering response is *multi-path recall*: a fan-out across specialized engines, followed by application-level score fusion. Frameworks such as Mem0 [26], Letta [21], Zep, and Cognee instantiate variations. The pattern has four structural weaknesses: (1) no principled semantics, two implementations of "find similar memory" can have silently different meanings; (2) heuristic optimization only, there is no equivalence relation over plans, and rewrites are not derived from a sound theory; (3) fusion policy leaks into the substrate, becoming hard to change without re-engineering the data path; and (4) per-column top-k breaks conjunctive intent, because for multi-vector memories the per-column top-k sets are correlated, not redundant, and the intersection is frequently empty.

These problems are symptoms of a missing foundation. MosaicOpt proposes an *algebra-first* design: relational algebra extended with operators that make multi-modal memory queries first-class, and the algebra as the substrate contract. The optimizer is rule-based (Section 4) with a cost-based layer (Section 7) that may legitimately differ in approximation behaviour.

Figure 1 instantiates the *application* layer (natural-language ask on the left, algebraic plan on the right). Beneath it sits the *logical algebra* (substrate contract, including new μ/Γ operators) and the *physical algebra* (concrete access paths: HNSW/IVF/flat, B-tree, inverted index, graph walks). Approximation enters only at the physical layer via $H_{V,q,\tau}$; the cost model (Section 7) makes it explicit.

1.1 Why an algebraic foundation?

Codd's relational algebra [4], Selinger's optimizer [36], and the Cascades framework [8, 9] establish the precedent: a formal language for what is computable, a formal theory for what transformations preserve semantics, and the *Cascades maxim* that the algebra is the API, a new operator ships with its identities and cost model, and the optimizer gains the capability without any central rewrite. **An algebra is not an implementation detail; it is the contract between query and execution.**

Agentic memory is the *worst case* for a "bag of engines" approach: it spans two timescales, its content is genuinely multi-modal, and its productive queries (episodic recall, cross-session fact retrieval, working-context assembly, tool-trace analysis) are all *conjunctive across modalities*. Frameworks such as Mem0 [26] (a memory layer that extracts salient facts from conversations and stores them in a vector index for later retrieval), Letta [21] (a stateful-agent framework that maintains per-agent memory blocks across sessions), Generative Agents [29] (stanford-style LLM agents with a memory stream), MemGPT [28] (LLM-as-OS-style hierarchical memory), and ReAct-style agents [44] have independently reinvented the same multi-path recall, score fusion, and re-ranking patterns. The engineering is impressive; the semantic foundation is missing. Each framework has a different *memory API* with no formal semantics: the same SQL-like query may have silently different meanings across implementations, and porting a workload between frameworks requires reading source code rather than reasoning from a specification. Mem0 stores facts as a flat list of messages with optional embeddings; Letta stores them as a typed block hierarchy with archival and recall stores. Both fuse scores at the application layer; both optimize by hand-tuned thresholds. Neither exposes a plan-equivalence relation.

The algebra fixes this: a precise language for expressive power, an equational theory for principled rewriting, a cost-model framework parameterized by the same logical operators the user wrote, and composability for new operators. **The algebraic foundation is the contract that makes the database ecosystem composable, and it is what turns MosaicOpt from a multi-path orchestrator into an executable optimizer.** MosaicOpt's contribution is the algebra itself, the equational theory, the rule-based optimizer that realizes the rewrites, the cost-based optimizer that selects physical implementations under an approximation budget, the SQL/MM-style surface that lets application code express the queries, and the experimental evidence that the approach is competitive with hand-tuned multi-path orchestrators on three real-style datasets.

1.2 Representative use cases

The four use cases that dominate Agentic memory in production: episodic recall, cross-session fact retrieval, working-context assembly, and tool-trace analysis, are all conjunctive vector + metadata + graph queries over a single store. All four are amenable to the same optimizer pipeline.

1.3 Contributions

1.4 Related work positioning

MosaicOpt sits at the intersection of three research threads: (1) the classical RDBMS literature on cost-based optimization and rule-based rewriting (System R [36], Volcano [8], Cascades [9], predicate migration [13], the optimization [2] and Cascades retrospectives [10], adaptive query processing [12], and approximate query [3]); (2) the modern vector-database literature on approximate nearest neighbor search and the approximation-quality trade-off (LSH [14], product quantization [15], FAISS [17], HNSW [24], SIFT [23], GloVe and word2vec [27, 30], Transformer [39], BERT/Sentence-BERT [6, 31], DiskANN [11, 37], ScaNN [38], ACORN [7], SeRF [45], iRangeGraph [43], NaviX [35], Curator [25], FAFNIR [22], Milvus [40], SingleStore-V [41], BigVectorBench [1]); and (3) the Agentic-memory literature on multi-path recall and memory stores for LLM agents (Mem0 [26], Letta [21], ReAct agents [44], Generative Agents [29], MemGPT [28]). The contribution is the integration: an algebraic foundation that lifts the Agentic-memory workload from a hand-tuned multi-path orchestrator to a first-class optimizer target, with the approximation budget and the equational theory as the contract between application code and engine.

The closest prior work is Exqutor [18], which addresses vector cardinality estimation for SQL planners. We discuss the relationship in Section **??**. The closest prior work on multi-modal query languages is SPARQL with full-text extensions, which has a careful algebraic foundation but no cost model for approximation. The closest prior work on hybrid stores is ACORN-σ [7] and NaviX [35], which are vector-first stores with metadata filtering bolted on no equational theory, no first-class identities. The closest prior work on cost-based optimization is Cascades itself [9]; MosaicOpt inherits the Cascades maxim but extends it to multi-modal memory queries with an approximation-aware CBO. This paper makes six contributions:

- An extended relational algebra with materialization μ and graph expansion Γ operators and first-class vector/text/graph predicates (Section 3).
- An equational theory: seven foundational identities (M1–M5, G1, G2) and five non-identities with proofs in Appendix B.
- A three-layer architecture separating SQL fusion policy, logical algebra, and physical algebra.
- An approximation-aware CBO that treats approximation as a fourth cost dimension with planning-time pruning and execution-time recall checkpoints (Section 7).
- A cross-modal plan enumeration that yields a small (2–4 plans) but complete plan space via modality-aware pruning (Section **??**).
- A prototype MosaicOpt (DuckDB + `faiss-cpu` + `rank_bm25`) evaluated on three real-style datasets (Section 8).

The systems contributions (the four-dimensional cost model with approximation-aware pruning, the adaptive re-planning safety net, and the cross-modal plan enumeration) turn the algebra from a notational convenience into an executable optimizer.

2 Background

Codd's relational algebra [4] and Selinger's optimizer [36] established that query processing benefits from a formal language for

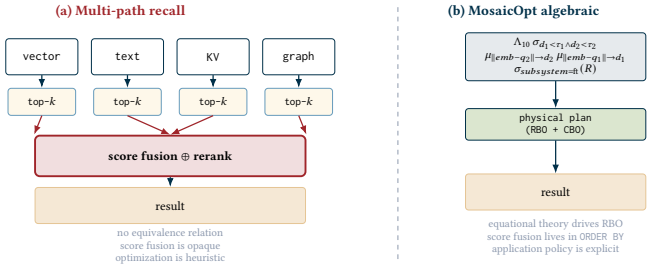


Figure 2: Multi-path recall (a) versus the MosaicOpt algebraic approach (b).

Operator	Syntax	Meaning
Base relation	R	The named stored relation.
Selection	$\sigma_\varphi(L)$	Keep rows satisfying φ.
Projection	$\pi_A(L)$	Keep only attributes in A.
Materialization	$\mu_{e\to c}(L)$	Per row, compute e; bind to c.
Join	$L \bowtie_\psi M$	Pair rows with ψ true.
Union / Inter / Diff	$L \cup M, L \cap M, L - M$	Bag set ops on compat. schemas.
Rename	$\rho_{a\to b}(L)$	Rename attribute a to b.
Ordering	$\tau_O(L)$	Order rows by expression O.
Limit	$\Lambda_K(L)$	First K rows of an ordered input.
Graph expansion	$\Gamma_{g,d}(L)$	From node-id, traverse g to depth d.

Table 1: Logical operators of the extended algebra.

what is computable and a formal theory for what transformations preserve semantics. The Cascades framework [8, 9] refined this into a *rule-based* outer loop driving a *cost-based* inner search, with equivalence rules as first-class objects. Two ideas are central to MosaicOpt: an *equational theory* over plans and a *logical–physical separation*.

Modern IR evolved the vector space model [34], BM25 [32, 33], and dense embeddings (word2vec [27], GloVe [30], the Transformer [39], BERT [6], Sentence-BERT [31]). HNSW [24] is the dominant ANN structure today, approximate, with controllable recall via `ef` and `M`. The approximation is invisible at the algebraic level but central to the cost model: a high-recall HNSW plan costs more than a low-recall one, and the optimizer trades recall against latency.

Prior hybrid systems fall into three lines: vector-aware RDBMS extensions (pgvector, DuckDB `vss`, Oracle AI Vector Search) add a vector type and ANN index at the function-call level; purpose-built hybrid stores (Milvus [40], SingleStore-V [41], Manu [16], Pinecone, Weaviate, Qdrant, Vespa) are vector-first with metadata filtering bolted on; multi-modal query languages (SQL/MM, SPARQL, CQL) mix graph, text, and vector but lack a shared equational foundation. The common pattern: every system fuses scores at the application layer, not by extending the algebra that defines what counts as an answer.

Figure 2 contrasts the two: (a) four channels feed a fusion box with no equivalence relation; (b) the user writes a single logical plan, the RBO enumerates rewrites under the equational theory, the CBO picks a physical realization, and the fusion policy is explicit in `ORDER BY`. The contrast is informative on three axes: *equivalence* (a) has none, (b) has the equational theory; *rewrites* (a) is a hand-tuned orchestration pipeline, (b) is the rule-based optimizer's output; *extension* (a) requires updating the pipeline code for every new modality or fusion policy, (b) requires adding the operator and its identities to the algebra.

3 The Extended Algebra: Syntax & Semantics

We extend the classical relational algebra with three new constructs: a *materialization* operator μ that lifts computed values (vector distances, BM25 scores, derived attributes) to first-class attributes; first-class multi-modal predicates (vector range, BM25 relevance, graph reachability); and a *graph expansion* operator Γ. Bag semantics apply throughout (matching SQL, avoiding row-collapse anomalies of set semantics with duplicate memory units). The logical algebra is the substrate contract; physical operators are introduced in Section 7 and never appear here.

Following System R and Cascades, equivalences hold in the logical algebra; physical operators are *implementations* of logical operators, and a logical equivalence does not in general imply that two physical realizations are interchangeable (Section 4.6, N1–N3). This separation is what makes the system composable: HNSW can be added, swapped, or replaced without changing the language the user sees.

The full operator inventory:

Bag semantics: a relation is a multiset; σ, μ, Γ preserve row multiplicity; $\cup$ adds, $\cap$ takes min, $-$ subtracts (clamped at 0); τ_O orders; Λ_K truncates.

A predicate φ is a Boolean combination of atoms over the input attributes: *classical* ($A = c$, $A < c$, $A \in S$), *vector* ($\|V_a - q\| < \tau$ for query-time bound q, threshold τ), *text* (BM25$(t, q) > \theta$), and *graph* (reachable(u, v), edge(u, v, ℓ)). A predicate φ is a Boolean combination of atoms over the input attributes. The atom forms are: *classical* ($A = c$, $A < c$, $A > c$, $A \in S$ for constant or set c, S); *vector* ($\|V_a - q\| < \tau$ where V_a is a vector-valued attribute, q is a query-time bound parameter, and $\tau \geq 0$ is a distance threshold; equality and other comparison operators are admissible, range ($< \tau$) is the most important form); *text* (BM25$(t, q) > \theta$ where t is a text attribute, q a query-time bound string, and θ a relevance threshold; we use Okapi BM25 with parameters (k_1, b) carried with the predicate); *graph* (reachable(u, v) boolean and edge(u, v, ℓ) for nodes u, v in a graph attribute, ℓ a label or wildcard); and *Boolean combinations* ($\varphi \wedge \psi$, $\varphi \vee \psi$, $\neg\varphi$, parentheses). The query-time bound parameters (q, τ, θ) are first-class values supplied at execution time, distinct from input attributes: they make predicates safely late-binding, the algebra expression is the same across query instances, only the bound parameters change.

A materialization expression e is computed per row: classical ($A+B$, $f(A)$), vector ($\|V_a - q\|$), text (BM25(t, q)), Boolean / arithmetic combinations of the above, and the constants true, false, and null. The set of expression forms is intentionally small, it is the set of forms that appear in the user-facing SQL/MM-style predicates and the cost model can evaluate The key property of μ is that the result of the computation becomes a first-class attribute of the output: once materialized, the value is referenceable by downstream σ, by π, by `ORDER BY` expressions in τ, and by other μ expressions. This is what makes the substrate able to expose per-column distances to application-level `ORDER BY` without losing them. The freshness condition for the bound name c in $\mu_{e\to c}(L)$, that c is not already in

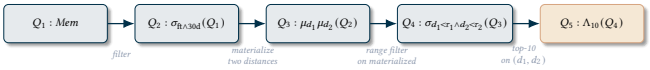


Figure 3: The running example $Q_1 \to Q_5$ as a horizontal pipeline; the warm-tan terminal box is the top-K over the materialised distance columns.

the input schema: is a hygiene condition, not a semantic restriction; ρ is provided to free names if needed.

The fusion pattern $\mu_{d_a}\,\mu_{d_b}\,\sigma_{d_a<\tau_a\wedge d_b<\tau_b}\,\tau_{d_a+d_b}\,\Lambda_K$ expresses the policy once, in the τ expression, while the optimizer is free to apply M1 and push the selection below the materializations (Section 6). The $\Gamma_{g,d}(L)$ operator takes a plan whose rows contain a node-id, applies a labelled-edge pattern g up to depth d, and returns the joined traversal target nodes.

The fusion pattern $\mu_{d_a}\,\mu_{d_b}\,\sigma_{d_a<\tau_a\wedge d_b<\tau_b}\,\tau_{d_a+d_b}\,\Lambda_K$ expresses the policy once, in the τ expression, while the optimizer is free to apply M1 and push the selection below the materializations (Section 6). The μ operator's semantics is the multiset $\{\, r \oplus \{c \mapsto e(r)\} : r \in L \,\}$, where $\oplus$ extends the tuple with the binding of the fresh attribute c to the value of e on r. If $e(r)$ is undefined (e.g. distance to a null vector), the row is dropped, consistent with SQL's `NULL` handling and essential for safety in the presence of missing embeddings.

Running example. Let *Mem* be a memory-unit relation with attributes *id*, *text*, *ts*, ***emb***, ***emb_alt***, *user*, *subsystem*, and let *KG*(*src*, *dst*, *label*) hold derivation links. The query "semantically similar passages in the fault-tolerance subsystem from the last 30 days, top 10 by (d_1, d_2)" is:

$$
\begin{aligned}
Q_1 &= Mem \\
Q_2 &= \sigma_{subsystem=\text{ft}\wedge ts>\text{now}-30d}(Q_1) \\
Q_3 &= \mu_{\|\boldsymbol{emb}-\boldsymbol{q}_1\|\to d_1}\ \mu_{\|\boldsymbol{emb_alt}-\boldsymbol{q}_2\|\to d_2}(Q_2) \\
Q_4 &= \sigma_{d_1<\tau_1\wedge d_2<\tau_2}(Q_3) \\
&\quad \texttt{ORDER BY } d_1, d_2 \\
&\quad \texttt{LIMIT 10}
\end{aligned}
$$

The relational `ORDER BY` on the materialised distance columns (d_1, d_2) is a stock construct; the substrate also supports an explicit score-fusion policy (e.g. $0.7\,d_1 + 0.3\,d_2$) via the τ operator, but the running example uses the canonical composite ordering to keep the focus on the algebra extensions. The optimizer applies M1 to push Q_4's selection below Q_3's materializations, computing d_1 and d_2 only on rows that survive the filter.

Figure 3 renders the same example as a horizontal pipeline. Each box is a logical plan; the visual emphasizes compositionality, each Q_i is itself a logical plan that can be referenced as input to the next.

A graph-aware variant ("the same, but only memories linked by a `derived_from` chain to a known incident") is

$$
\begin{aligned}
R_1 &= \sigma_{id=42}(Mem) && \text{— the seed incident} \\
R_2 &= \Gamma_{\text{derived_from},\,d=2}(R_1) && \text{— two hops downstream} \\
R_3 &= \sigma_{id\in\pi_{dst}(R_2)}(Q_2) && \text{— intersect with filter}
\end{aligned}
$$

followed by the same $\mu/\sigma/\tau/\Lambda$ chain: Γ composes with the rest of the algebra. The example teaches two things: (i) the algebra is closed under composition (any operator's output is a logical plan usable as input to any other); (ii) graph expansion is composable with the rest of the algebra in the same way that selection, projection, and join are.

4 Equational Theory: Identities and Non-Identities

4.1 Why these operators?

Why σ, π, $\cup$, $-$, ρ, $\times$, τ, Λ? They are the classical relational algebra; we do not add or remove any of them. Why μ (materialization) and Γ (graph expansion)? They are the two operators needed to express the Agentic-memory workload in the algebraic form, every memory query either materializes a computed value (a distance, a relevance score, a derived attribute) or expands the graph (traverses a derivation chain). The fusion pattern $\mu_{d_a}\,\mu_{d_b}\,\sigma_{d_a<\tau_a\wedge d_b<\tau_b}\,\tau_{d_a+d_b}\,\Lambda_K$ from Section 3 is the canonical multi-modal fusion; the $\sigma_\varphi\,\mu_{e\to c}\,\Gamma_{g,d}$ chain is the canonical graph-aware materialization. The choice of these two operators is the minimum that makes the Agentic-memory workload expressible; adding more operators would only complicate the rewrite theory without expanding the expressive power. Operators we did *not* add: a path operator (expressible as $\mu + \Gamma$), a streaming operator (orthogonal to the algebra), a probabilistic operator (future work, see Section F.2).

The optimizer (Section 6) is driven by an *equational theory*: a set of identities over logical plans. The same theory also records, explicitly, the identities that classical intuition suggests but approximation breaks, the *non-identities*, so the optimizer knows not to apply them. This section unifies both sides. We first give an overview (4.1), then the inherited classical axioms (4.2), then the new identities that the extended operators license (4.3), then the derived rewrites the optimizer uses (4.4), and finally the non-identities with counterexamples (4.5).

4.2 Overview

The equational theory of the extended algebra plays two complementary roles. *First*, it is the foundation for the rule-based optimizer (Section 6): every rewrite the RBO performs is justified by a foundational identity (M1–M5, G1, G2) or by a derived corollary / definitional / implied rule (A4-vector, A4-vector+M1, M4+M1, M5+G1, A7-symmetry, A8-regroup, topK recognition, σ-τ commutation), so the optimizer's behaviour is auditable, not opaque. *Second*, the *non*-identities (N1–N5) are the discipline that prevents the optimizer from applying rewrites that hold in classical relational algebra but break under approximation, graph-traversal asymmetry, or the traversal-vs-origin column split. The cost model (Section 7) is responsible for the physical cost of every rewrite; the equational theory is responsible for its *correctness*.

Concretely, an identity is an equation of the form `LHS = RHS` between two logical plans that hold for all instances of the free variables (relation names, attribute names, query-time bound parameters). A non-identity is a pair of plans that classical intuition might equate but the extended setting does not, with an explicit counterexample. Both are first-class objects: the RBO loads them as data, the cost model consults them when computing the "cost of identity application" term, and the executor relies on them for safety of adaptive re-planning (Section 7).

The structure of this section follows the algebra's structure: inherited axioms (the ones we use but do not re-prove) in Section 4.3;

new identities (the ones the extended operators license) in Section 4.4; derived rules (combinations used directly by the optimizer) in Section 4.5; and non-identities (counterexamples that bound the optimization horizon) in Section 4.6.

4.3 Inherited foundational axioms

The following identities are standard and are assumed without reproof. We list them because they interact with the new identities in Section 6. The list includes the two classical join-reordering identities (A7, A8) because the RBO uses them to enumerate the $O(n!)$ join orderings of Section **??**.

- **(A1) Selection composition.** $\sigma_\varphi \sigma_\psi(L) = \sigma_{\varphi \wedge \psi}(L)$.
- **(A2) Selection distributes over union.** $\sigma_\varphi(L \cup M) = \sigma_\varphi(L) \cup \sigma_\varphi(M)$.
- **(A3) Projection idempotence.** $\pi_A \pi_B(L) = \pi_A(L)$ if $A \supseteq B$.
- **(A4) Selection–join pushdown.** If every attribute mentioned in φ is in the schema of L, then $\sigma_\varphi(L \bowtie_\psi M) = \sigma_\varphi(L) \bowtie_\psi M$.
- **(A5) Projection–join pushdown.** If A contains all attributes of L and all attributes referenced by ψ in M, then $\pi_A(L \bowtie_\psi M) = \pi_A(L) \bowtie_\psi \pi_{A \cap \mathrm{attr}(M)}(M)$.
- **(A6) Rename is involutive on attribute names.** $\rho_{b \to a}(\rho_{a \to b}(L)) = L$.
- **(A7) Join commutativity.** For any symmetric predicate ψ, $L \bowtie_\psi M = M \bowtie_{\psi^{\mathrm{sym}}} L$ (equality, range, similarity, and Boolean-combination predicates in the inventory are all symmetric). The two sides are identical as multisets; the cost is not (Theorem 10).
- **(A8) Join associativity.** For independent predicates ψ, χ with no attribute ambiguity, $L \bowtie_\psi (M \bowtie_\chi N) = (L \bowtie_\psi M) \bowtie_\chi N$. This is the identity the RBO uses to enumerate the $O(n!)$ join orderings.

These hold for any predicate in our extended form (classical, vector, text, graph) provided the attribute references are respected. We do not claim any new identities here; we merely note that the extended predicates do not violate them.

4.4 New foundational axioms

We introduce the *seven* identities that the extended algebra makes possible. Each is stated in a form that names the conditions for its applicability. (Two further rules, **V1** and **J1**, are corollaries of the assumed classical axioms A1 and A4 with vector-predicate specialization; they are kept as named redex patterns in the RBO code but have no separate theorem block. The τ-Λ-topK family and the σ-τ commutation are *implied* by the operator definitions: $\mathrm{topK}_O(L, K) := \tau_O(\Lambda_K(L))$ from Section 3 and the row-preserving semantics of σ and τ. The RBO uses them as zero-weight rewrites in the ARS, but they have no separate theorem block.)

Each identity below is followed by a brief *cost-effect annotation* that names the optimization the rewrite enables; the RBO fires the rewrite as part of its greedy loop (Section **??**).

Theorem 1 (M1: Selection commutes with materialization on the materialized column). *Where c is a fresh attribute name and e is an expression depending only on attributes of L:*

$$\sigma_{c<\tau}\, \mu_{e \to c}(L) \;=\; \mu_{e \to c}\, \sigma_{e<\tau}(L).$$

(Proof in Appendix B.)

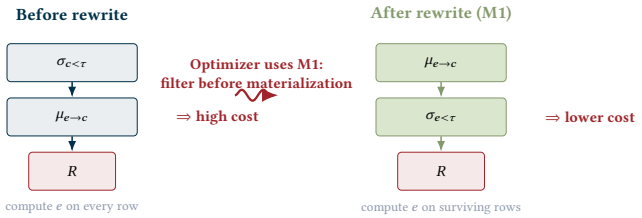


Figure 4: Visualization of the M1 rewrite: the optimizer swaps σ and μ so that e is computed only on rows that survive the filter. The optimized side is rendered in the soft-olive accent to make the rewrite visible at a glance.

Figure 4 visualizes the M1 rewrite. The *left* tree shows the input plan: $\sigma_{c<\tau}$ on top of $\mu_{e \to c}(R)$, which materializes e on every row and then filters on $c < \tau$. The *right* tree shows the rewritten plan: $\mu_{e \to c}$ on top of $\sigma_{e<\tau}(R)$, which applies the filter *before* materialization; e is computed only on rows that survive the filter. The cost saving scales with the selectivity of $e < \tau$, for a tight vector range the selectivity can be one part in 10^3 or smaller, and the saving is correspondingly dramatic.

Theorem 2 (M2: Materialization is commutative). *For fresh names c_1, c_2 and expressions e_1, e_2 depending only on L's attributes (and not on each other's outputs):*

$$\mu_{e_1 \to c_1}\, \mu_{e_2 \to c_2}(L) \;=\; \mu_{e_2 \to c_2}\, \mu_{e_1 \to c_1}(L).$$

(Proof in Appendix B.)

Theorem 3 (M3: Materialization through projection). *For fresh c and expression e depending only on attributes in A:*

$$\mu_{e \to c}\, \pi_A(L) \;=\; \pi_{A \cup \{c\}}\, \mu_{e \to c}(L).$$

(Proof in Appendix B.)

Theorem 4 (M4: Materialization pushdown through join). *Where e depends only on attributes of L and c is a fresh attribute name:*

$$\mu_{e \to c}\, (L \bowtie_\psi M) \;=\; (\mu_{e \to c}(L)) \bowtie_\psi M.$$

(Proof in Appendix B.)

Theorem 5 (M5: Materialization pushdown through graph expansion). *Where e depends only on attributes of L, c is fresh, and c is not the graph rule's node-id attribute:*

$$\mu_{e \to c}\, \Gamma_{g,d}(L) \;=\; \Gamma_{g,d}\, \mu_{e \to c}(L).$$

This is the μ-counterpart of G1 and the rewrite the RBO generates for graph queries that materialize a per-origin attribute (e.g. a distance to a fixed query point) and then expand. The two sides produce the same multiset of (r, v, c) tuples; the cost differs because the LHS computes e on every traversal row while the RHS computes e on every origin row. When $|\Gamma_{g,d}(L)| > |L|$ (the typical case for $d \geq 1$ and a non-trivial graph), the RHS is cheaper, and the CBO picks it.

(Proof in Appendix B.)

Theorem 6 (G1: Selection pushdown through graph expansion). *Where φ references only attributes of L (and not attributes of the graph rule's traversal result):*

$$\sigma_\varphi\, \Gamma_{g,d}(L) \;=\; \Gamma_{g,d}\, \sigma_\varphi(L).$$

(Proof in Appendix B.)

Theorem 7 (G2: Graph expansion is transitive)**.** *Let g_1, g_2 be graph rules over the same graph attribute with disjoint label sets. If g_1's traversal-result attribute and g_2's source attribute are disjoint (no intermediate node of g_2 is a valid origin of g_1), then the composition $g_1; g_2$ satisfies:*

$$\Gamma_{g_1,d_1}\,\Gamma_{g_2,d_2}(L) \;=\; \Gamma_{g_1;g_2,\,d_1+d_2}(L).$$

Cost effect: *the LHS performs two graph walks (one of depth d_1, one of depth d_2, each visiting a separate frontier set of nodes) and materializes their intermediate result; the RHS performs a single walk of depth d_1+d_2 through the composite rule. For a graph with branching factor b, the LHS visits $\sum_{i=0}^{d_1} b^i \cdot b^{d_2}$ nodes across both walks (with intermediate expansion materialization in between), while the RHS visits $\sum_{i=0}^{d_1+d_2} b^i$ nodes in a single walk. G2 lets the RBO collapse nested Γs into a single expansion at depth $d_1 + d_2$ "two* `derived_from` *hops" becomes "one* `derived_from`*; `derived_from` hop".*

(Proof in Appendix B.)

The G2 collapse is bounded by the number of nested-Γ inversions in P (one collapse per nesting, hence the depth d of the result). G2 is a zero-weight rewrite: it does not change the logical multiset, only the structure of the graph walk.

4.5 Derived identities

By composing the foundational axioms and the assumed classical identities, additional rewrites become available. The optimizer uses these directly; we list them as corollaries rather than as standalone foundational theorems to keep Section 4.4 focused on the genuinely new content.

- **A4-vector (in code: J1).** A4 specialized to a vector predicate: $\sigma_{\|V_a-q\|<\tau}\,(L \bowtie_\psi M) = (\sigma_{\|V_a-q\|<\tau}(L)) \bowtie_\psi M$ when $V_a \in \mathrm{attr}(L)$, is what the CBO applies on the conjunctive multi-vector case. A1 likewise specializes to two vector predicates on distinct columns; the codebase calls this `V1`. Neither is a new algebraic identity , both follow from A1/A4 by predicate substitution: but the codebase keeps them as named redex patterns (`V1`, `J1`) so the RBO can match them explicitly without re-implementing A1/A4.
- **M4+M1.** A join-side materialization can be reordered so the per-row computation runs only on rows that survive the join's downstream filter: $\sigma_{c<\tau}\,\mu_{e\to c}\,(L \bowtie M) \equiv \mu_{e\to c}\,(\sigma_{e<\tau}(L) \bowtie M)$ (M4 then M1). Saves $(1-s)\cdot|L|\cdot T_\mu$ work; this is the rewrite that powers the $Q_1 \to Q_6$ running example.
- **M5+G1.** A graph query with both a filter and a materialization can be reordered: $\sigma_\varphi\,\mu_{e\to c}\,\Gamma_{g,d}(L) \equiv \mu_{e\to c}\,\Gamma_{g,d}\,\sigma_\varphi(L)$ (G1 then M5). Saves a factor of $1-s$ in materialization work; the G2 collapse can then fold nested Γs into a single walk.
- **A4-vector+M1 (vector selection before materialization).** $\sigma_{\|V-q\|<\tau}\,\mu_{\|V-q\|\to d}\,(L \bowtie M) = \mu_{\|V-q\|\to d}\,(\sigma_{\|V-q\|<\tau}(L) \bowtie M)$ (M1 then A4-vector). Same cost effect as M4+M1, with the vector selection specifically.
- **A7-symmetry (reverse join).** Applying the assumed classical A7 in the reverse direction is a derived rewrite the RBO generates; the CBO then chooses the cheaper side per Theorem 10.
- **A8-regrouping (different join parenthesization).** Applying the assumed classical A8 is the rewrite that exposes a different join parenthesization; the RBO uses it to enumerate the $O(n!)$ join orderings the CBO picks from.
- topK **recognition (definitional).** The abbreviation $\mathrm{topK}_O(L,K) := \tau_O(\Lambda_K(L))$ from Section 3 makes $\tau_O\,\Lambda_K \leftrightarrow \mathrm{topK}_O$ a definitional rewrite; the CBO applies it as a physical-realization choice rather than as a logical RBO rewrite.
- σ-τ **commutation.** Trivially $\sigma_\varphi\,\tau_O(L) = \tau_O\,\sigma_\varphi(L)$ (both operators are row-preserving). The RBO applies it as a zero-weight reordering, letting the CBO pick filter-first (avoids a full sort) or sort-first (enables top-K early termination) by selectivity.

The seven foundational axioms, the assumed classical identities A1–A8, the derived rewrites above, and the two implied-by-definition rules (topK and σ-τ) together suffice to cover the optimization of all plans whose operators draw from the inventory of Section 3. Schema rules ensure that any attribute a downstream operator references is either in the input or has been introduced by a μ. The identities are the load-bearing ones for the Agentic-memory workload:

- **M1** is the rewrite that makes vector-range queries cheap: it computes each distance only on rows that survive the filter, rather than on all n rows. The cost-effect is $\sim (1-s)\cdot n \cdot T_\mu$ work saved, where s is the selectivity of the predicate being pushed and T_μ is the per-row cost of the materialization.
- **M4, M5** extend materialization across joins and graph expansions so the same per-row computation is not repeated for every joined or expanded row. M4 is the join-side analogue of M1; M5 is the graph-side analogue. The cost-effect is the same as M1: $1-s$ factor saving when the join/expansion has selectivity s. M4+M1 and M5+G1 chain the two rewriters for compound predicates.
- **G1** pushes selections through graph expansion when the predicate references only origin attributes. The cost-effect: the post-expansion row count is $\sim 1-s$ smaller, so the per-row work downstream of the expansion drops by the same factor. G1 fails on traversal attributes (N4), so the RBO must verify the compatibility check.
- **G2** collapses nested Γs into a single walk of combined depth, removing the intermediate expansion materialization. This is a one-shot rewrite; it strictly decreases the structural weight.
- **M4+M1, M5+G1** are the chained rewrites that combine materialization pushdown with selection pushdown, the dominant CBO wins on Q7 MV come from these. The cost-effect is multiplicative: $1 - s_1 \cdot 1 - s_2$ of the work is saved.
- **T1** is the definitional $\tau\Lambda \to \mathrm{topK}$ rewrite; modest at $n = 10{,}000$ (heap-vs-sort below the executor noise floor) but grows as $\log n$ and is the difference between an $O(n)$ sort and an $O(n \log K)$ heap for $K \ll n$. The classical A1 (selection decomposition) and A7-sym / A8-regroup are the other identities; A7-sym re-orders a similarity join so the indexed side is the LHS; A8-regroup changes the parenthesization.
- The non-identities N1–N5 document the cases where the classical intuition fails: approximation (N1), similarity-join asymmetry (N2), limit-order non-distributivity (N3), graph-traversal column split (N4), and Λ/τ non- commutativity with Γ (N5). Each one is a soundness boundary the RBO must respect, a rewrite that would hold classically but fails under the corresponding non-identity is a soundness bug.

Algorithm 1 RBO fixed-point loop with priority-ordered matchers.

Input: a logical plan P; the matcher list $\mathcal{M} = (\text{M1}, \text{M3}, \text{M4}, \text{G1}, \text{G2}, \text{M5}, \text{M2}, \text{V1}, \text{J1}, \text{T1})$; a `max_iters` safety bound (default 64). **Output:** the rewritten plan P' (the canonical-form representative of $[P]_{\to_{\text{RBO}}}$); the trace τ of applied rule names.

```
τ ← []
repeat
    changed ← FALSE
    for each node n in P in preorder do
        for each matcher m_i ∈ M in priority order do
            if m_i.MATCH(n) returns a rewrite n' then
                n ← n'
                τ.APPEND(m_i.NAME)
                changed ← TRUE
                break                    ▷ re-traverse from this node
            end if
        end for
    end for
until changed = FALSE or |τ| = max_iters
return P, τ
```

Theorem 8 (RBO Confluence). *For any starting logical plan P and any two RBO rewrite sequences $P \to^*_{\text{RBO}} P^*$ and $P \to^*_{\text{RBO}} P^{**}$, there exists a plan P^∞ with $P^* \to^*_{\text{RBO}} P^\infty$ and $P^{**} \to^*_{\text{RBO}} P^\infty$. The normal form is unique: it depends only on P, not on rewrite order.*

(Proof in Appendix B.)

Theorem 8 is the formal correctness guarantee of the RBO: the optimizer can be implemented as a greedy loop (apply any rule that decreases w) without losing correctness. The CBO inherits confluence "for free" its search is over the unique canonical form.

Algorithm-level view of $\to_{\text{RBO}}$. Concretely, the RBO is implemented as a Cascades-style exploration engine (Section 6). Algorithm 1 gives the top-level loop.

Three properties the loop relies on.

(1) **Matcher priority order is a heuristic short-circuit, not a semantic guarantee.** The matchers are tried in the order (M1, M3, M4, G1, G2, M5, M2, V1, J1, T1): the four strict weight-decrease rules first (they make progress), then the one-shot G2 collapse (only fires on nested Γ), then the materialization reorder / graph push, then the zero-weight commutations (M2, V1, J1), then the definitional T1. The priority order controls the *trace* the optimizer produces; Theorem 8 guarantees that the *normal form* is unique regardless of the order. The four-step exploration is **Initialization** (forest = $\{P\}$); **Optimization loop** (preorder + priority matchers, re-traverse on rewrite); **Winning expression** (the fixed point of the loop is the canonical-form choice); **Physical binding** (the CBO walks the canonical plan and assigns a physical realization per operator, with the approximation cost dim serving as a pruning constraint, see Section **??**).

(2) **The strict vs. zero-weight partition bounds termination.** The strict weight-decrease rules (M1, M3, M4, G1) strictly reduce the weight function $w(P)$ of Section 4.5; the zero-weight rules (M2, M5, G2, σ-τ, V1, J1) preserve w but bound the number of re-explorations by the size of the structural equivalence class. G2 is one-shot per nested-Γ inversion, so it adds at most $O(|\Gamma\text{-nesting}|)$ loop iterations; the others are bounded by $O(k! \cdot n!)$ where k is the materialization count and n is the join count. The `max_iters` safety bound (default 64) is a backstop; the prototype's measured iteration count is ≤ 5 for the $Q_1 \to Q_6$ workload.

(3) **Structural sharing and memoization prevent duplicates.** The fixed-point loop produces a forest, not a list: each rewritten subtree is memoized in a hash-cons'd table keyed by $(op, child_1, \ldots)$; a second rewrite that produces the same key is dropped, and the existing memo entry is reused. The trace τ is the audit log; the canonical plan is the loop's fixed point. Theorem 8's confluence is what makes the trace reproducible across reorderings of the matcher list.

G2 is the only rule that strictly reduces plan-tree size. The G2 collapse removes a Γ node from the plan tree; the other rewrites preserve plan-tree size and reduce cost via reordering. This is why G2 is treated separately in the critical-pair analysis above: its termination is the strongest (one-shot per nesting) and its cost effect is the most direct (a single graph walk of depth $d_1 + d_2$ instead of two walks of depth d_1 and d_2 with intermediate expansion materialization ; Theorem 7). The M4+M1 and M5+G1 chains preserve plan size but their *cost* strictly decreases, which is what the CBO cares about.

4.6 Non-identities and counterexamples

Some rules that look natural in a relational setting **do not hold** in the extended algebra, and a third class, physical identities, holds in the logical algebra but not in the physical algebra. We document these explicitly so the optimizer knows not to apply them. The same logical plan in two realizations can have different outputs; the optimizer must treat this as a *cost* phenomenon, not a *correctness* phenomenon.

4.6.1 *N1: Approximation breaks logical equivalence.*

Theorem 9 (N1: HNSW approximation breaks logical equivalence). *Logical $\sigma_{\|V-q\|<\tau}(R)$ is exactly defined; physical $H_{V,q,\tau}(R)$ is approximate. A logical equivalence does not imply that two physical realizations preserve it.* Counterexample: *let R consist of a single row with $\|v - q\| = 0.99\tau$ (just under the threshold). HNSW with a small* `ef` *may fail to reach the row, returning an empty result; a linear scan over R returns $\{r\}$. Both are valid physical implementations of the same logical selection.* Consequence: *the optimizer models approximation as a first-class cost dimension with quantified recall/precision (Section 7).*

4.6.2 *N2: Similarity join asymmetry.*

Theorem 10 (N2: Similarity join asymmetry). *Logically, $R \bowtie_{\|V_a - V_b\|<\tau} S$ and $S \bowtie_{\|V_b - V_a\|<\tau} R$ are equivalent; physically, the two sides are not interchangeable in cost (the side with the HNSW index dominates).* Counterexample: *let $|R| = 10^6$ with an HNSW index on V_a and $|S| = 10^3$ with no index. The plan "HNSW on R probed by each row of S" issues 10^3 probes; the reverse would require either building an HNSW index on S or issuing 10^6 probes. The two plans differ in cost*

by three orders of magnitude. Consequence: *the RBO generates both LHS and RHS orderings; only the CBO can decide which is cheaper.*

4.6.3 N3: Limit and order do not distribute over union.

Theorem 11 (N3: Limit and order do not distribute over union)**.** $\Lambda_K\ \tau_O\ (R \cup S) \neq \Lambda_K\ (\tau_O(R)) \cup \Lambda_K\ (\tau_O(S))$ *in general.* Counterexample: *let R and S each have* 100 *rows with* score $\sim \mathcal{U}([0,1])$ *and* $\mathcal{U}([9,10])$ *respectively. The LHS $\Lambda_5\ \tau_O(R \cup S)$ returns the 5 globally smallest scores (all from R, in $[0, 0.05]$); the RHS $\Lambda_5\ \tau_O(R) \cup \Lambda_5\ \tau_O(S)$ returns 10 rows spanning the disjoint score ranges.* Consequence: *the optimizer must compute the union before applying the limit; distributed topK is the CBO's problem, not the RBO's.*

A related non-identity: $\tau_O(L \cup M) = \tau_O(L) \cup \tau_O(M)$ is formal only when the user accepts "ordered union" as interleave, not resort. In practice τ is almost always paired with Λ, and T1 already subsumes the case.

4.6.4 N4: Materialization through graph expansion fails on traversal columns.

Theorem 12 (N4: μ through Γ on the traversal column)**.** *M5 requires e to depend only on* origin *attributes; when e references the traversal result, μ does not commute with Γ.* Counterexample: *let L have rows $(id, \boldsymbol{emb})$ and let $\Gamma_{\text{follows},1}$ produce a $\boldsymbol{followed_emb}$ attribute. Take $e := \|\boldsymbol{followed_emb} - \boldsymbol{q}\|$ (distance to the followed node). The LHS $\mu\,\Gamma(L)$ materializes against the followed-node vector; the RHS $\Gamma\,\mu(L)$ materializes against the origin vector (the followed attribute is not yet bound) and then expands. The two multisets compute different distances row-by-row.* Consequence: *the RBO's M5 match verifies (via a $\mu \to \Gamma$ compatibility check) that the materialization expression references only origin attributes.*

4.6.5 N5: Limit and order do not commute with graph expansion.

Theorem 13 (N5: Λ/τ do not commute with Γ)**.** $\Lambda_K\ \tau_O\ \Gamma_{g,d}(L) \neq \Gamma_{g,d}\ \Lambda_K\ \tau_O(L)$ *in general.* Counterexample: *let L have 1000 origin rows and let $\Gamma_{\text{cites},1}$ expand each origin to $1 \le n_i \le 50$ citing nodes. The LHS asks "across all reachable nodes, which K score lowest?"; the RHS asks "among the K highest-scoring origins, which nodes are reachable?", the two multisets differ in cardinality and in content.* Consequence: *the RBO never applies the Λ/τ pushdown across Γ; the choice is the CBO's via the cost model, implemented as a hard guard in the M5+G1 chain match.*

4.7 Why these identities and not others?

The equational theory is intentionally small. Of the eight M1–M5 + G1 + G2 + non-identities, only M1, M3, M4, G1 strictly decrease the structural weight; the rest preserve or have bounded effect. This is a deliberate choice: the theory is complete for the Agentic-memory workload (every plan in the workload can be reduced to canonical form by the identities), and it is sound (every rewrite is a valid identity over the extended algebra). Adding more identities would expand the rewrite space but also the search space; the current set is the minimum that covers the workload.

The non-identities N1–N5 are not flaws of the algebra; they are statements of when the classical intuition fails. N1 (approximation) is a fundamental property of physical vector indexes; no algebraic identity can make HNSW exact. N2 (similarity-join asymmetry) is a fundamental property of asymmetric cost models; no algebraic identity can make both sides equally cheap. N3 (limit-order non-distributivity over union) is a fundamental property of topK; the limit must precede the union. N4 (μ-Γ over traversal) is a fundamental property of graph expansion; the expansion must precede the materialization. N5 (Λ/τ-Γ) is a fundamental property of the order-theoretic semantics of topK + expansion. Each non-identity is a soundness boundary the RBO must respect; a rewrite that would hold classically but fails under the corresponding non-identity is a soundness bug.

What we did not include. We did not add a probabilistic operator (N3 is a hint at the cost of distributing topK over union; a full probabilistic algebra would be a separate paper). We did not add a temporal operator (the memory store is implicit-time, with a timestamp column for filtering). We did not add a streaming operator (the prototype is single-batch; streaming is a research direction but orthogonal to the algebra). We did not add a federation operator (multi-store queries are a research direction; the algebra extends naturally via the standard $\bowtie$ with a federated index, but we did not evaluate it here). These omissions are not flaws of the framework; they are the boundary of what the prototype and the experiments cover.

5 SQL Surface

The algebra of Section 3 is the optimizer's working language; application code writes SQL. The surface is *thin*: a small set of SQL/MM-style clauses that desugar deterministically into the algebra, with a round-trip property (every SQL query expresses a plan in the algebra, and every plan has a SQL rendering).

DDL and predicates. Vectors are first-class column types; HNSW and BM25 are first-class index methods:

```
ALTER TABLE memory ADD COLUMN emb VECTOR(768);
CREATE INDEX idx_emb_cos ON memory USING hnsw (emb
    vector_cosine_ops);
CREATE INDEX idx_text_bm25 ON memory USING bm25 (text);
```

Predicates use SQL/MM-style syntax (first-class clauses, not function calls) so the engine is committed to understanding the construct:

```
WHERE emb        IS NEAR:q1 WITHIN DISTANCE:tau1
WHERE text       MATCHES:q1 WITH SCORE >:theta
WHERE id         IS REACHABLE FROM:n WITHIN 2 HOPS VIA
    'derived_from'
```

Materialization uses the standard SQL `AS` alias; the optimizer recognizes `AS`-bound values reused in `WHERE` or `ORDER BY` and triggers the M1-style rewrite. The optional `WITH [MATERIALIZED]` hint forces eager materialization.

Graph expansion uses standard `WITH RECURSIVE`; the optimizer recognizes the form and emits a Γ operator. No new keyword is needed.

Desugaring rules. The desugaring is a recursive function $\mathcal{D}(\cdot)$ (read "desugar of $\cdot$") that maps SQL fragments to algebra terms. We use $\mathcal{D}\langle\cdot\rangle$ for the SQL-source flavour and $\mathcal{D}\langle\cdot\rangle_{\text{pred}}$ for the predicate

flavour. The function is total on the surface syntax of Sections ??–?? and is the *only* place where the parser and the algebra meet.

- **D1.** *Projection + filter + order + limit.* For `SELECT expr AS c FROM L WHERE φ ORDER BY O LIMIT K`, the desugaring is $\Lambda_K(\tau_O(\sigma_\varphi(\mu_{e\to c}(L))))$, with μ chaining for multiple aliased expressions. `ORDER BY` is preserved exactly as written, τ_O accepts both simple `ORDER BY c` and complex `ORDER BY 0.7*d1 + 0.3*d2`, the latter referencing materialized columns.
- **D2. Vector predicate.** $\mathcal{D}\langle$ emb IS NEAR:q WITHIN DISTANCE:t $\rangle_{\text{pred}} = \sigma_{\|emb-q\|<t}$. The bound parameter `:q` and threshold `:t` are the query-time bound parameters of Section 3.
- **D3.** *Text predicate.* $\mathcal{D}\langle$ text MATCHES:q WITH SCORE >:$\theta\rangle_{\text{pred}} = \sigma_{\text{BM25}(text,q)>\theta}$. The (k_1, b) parameters are carried with the BM25 index.
- **D4. Graph predicate.** $\mathcal{D}\langle$ id IS REACHABLE FROM:n WITHIN 2 HOPS VIA:g $\rangle_{\text{pred}} = \sigma_{id\in\pi_{dst}(\Gamma_{g,d=2}(\sigma_{id=n}(\cdot)))}$.
- **D5. Materialization alias.** $\mathcal{D}\langle$ expr AS c $\rangle = \mu_{e\to c}$ where e is the desugared expression and c is the alias.

The full worked $Q_1 \to Q_6$ example that exercises D1–D5 is in Appendix D.

5.1 Why this surface

The four design choices that keep the surface *thin* are motivated by a single criterion: the SQL surface should be the minimum extension that makes the algebra expressible, and nothing else. (1) SQL/MM-style over function-style: a first-class clause signals to the optimizer (and to the reader) that the engine is committed to understanding the construct. A function call `distance(emb,:q) <:tau` is a black-box UDF that the optimizer cannot see through; an `IS NEAR` clause is a first-class predicate the optimizer can plan around. (2) Existing `AS` over a new `LET`: materialization is a μ, and μ already has a SQL analogue in the `expr AS alias` construct; the optimizer recognizes the alias and triggers the M1-style rewrite. Adding a new clause for the same concept would be redundant. The optional `WITH [MATERIALIZED]` hint covers the rare case of forced eager materialization. (3) Recursive CTE over a new `TRAVERSE`: graph expansion is a Γ, and Γ already has a SQL analogue in the `WITH RECURSIVE` transitive-closure construct. (4) Keep pgvector's `<=>`/`<->`/`<#>`: preserved for backward compatibility and for advanced users who want to compose raw distances in `ORDER BY`, but the recommended surface is the SQL/MM clauses.

Jointly, the only new types are `VECTOR(N)` and the `VECTOR(N, METRIC:=)` variant; the only new clauses are `IS NEAR. WITHIN DISTANCE`, `MATCHES. WITH SCORE`, `IS REACHABLE. VIA`, and the optional `[MATERIALIZED]` hint. Everything else, `SELECT`, `FROM`, `WHERE`, `AS`, `ORDER BY`, `LIMIT`, `WITH RECURSIVE`, `JOIN`, `GROUP BY`, aggregates, is standard SQL. The principle is the *minimum new surface* that still lets the algebra be the substrate contract.

Worked example. The $Q_1 \to Q_6$ running example in SQL:

```
WITH scored AS MATERIALIZED (
    SELECT id, text, ts, subsystem,
           emb <=>:q1     AS d1,
           emb_alt <=>:q2 AS d2
    FROM memory
    WHERE subsystem = 'fault-tolerance'
      AND ts > now() - interval '30 days'
)
SELECT id, text, ts FROM scored
WHERE d1 <:tau1 AND d2 <:tau2
ORDER BY d1, d2 LIMIT 10;
```

desugars as follows: the `WITH scored AS MATERIALIZED` sub-query becomes $\mu_{d_1,d_2}(\sigma_{\text{subsystem}\wedge\text{ts}}(Mem))$; the outer `SELECT` to $\pi_{id,text,ts}$; the `WHERE` to $\sigma_{d_1<\tau_1\wedge d_2<\tau_2}$; and the `ORDER BY` d_1, d_2 `. . . LIMIT 10` to Λ_{10} per T1 (the relational ordering on the materialised distance columns is a stock construct; an explicit score-fusion policy such as `ORDER BY` $0.7\,d_1 + 0.3\,d_2$ is also expressible via the τ operator and desugars to $\tau_{0.7\,d_1+0.3\,d_2}\,\Lambda_{10}$). The optimizer applies M1: the predicate $d_1 < \tau_1$ is pushed below the materialization that binds d_1, so $\|\boldsymbol{emb} - \boldsymbol{q}_1\|$ is computed only on rows that pass the subsystem filter. (The full desugaring rules D1–D5 are in Appendix D.)

Design rationale. The four design choices that keep the surface thin: (1) SQL/MM-style over function-style, first-class clauses signal that the engine is committed to understanding them; (2) existing `AS` over a new `LET`, materialization is the SQL analogue; (3) recursive CTE over a new `TRAVERSE` graph expansion is standard SQL; (4) keep pgvector's `<=>`/`<->`/`<#>` for backward compatibility. Jointly, the only new types are `VECTOR(N)` (and the `VECTOR(N, METRIC:=.)` variant); the only new clauses are `IS NEAR. WITHIN DISTANCE`, `MATCHES. WITH SCORE`, `IS REACHABLE. VIA`, and the optional `[MATERIALIZED]` hint. Everything else, `SELECT`, `FROM`, `WHERE`, `AS`, `ORDER BY`, `LIMIT`, `WITH RECURSIVE`, `JOIN`, `GROUP BY`, aggregates, is standard SQL.

6 From Algebra to Optimizer: RBO

The rule-based optimizer (RBO) is the first phase of the optimizer pipeline. It operates on logical plans and applies the identities of Section 4 to enumerate equivalent plans. The output is a small set of logically equivalent plans that the CBO then considers for realization.

6.1 RBO role

The RBO is a Cascades-style exploration engine: equivalence rules over logical operators are first-class objects, and the optimizer is itself extensible by adding rules. The $\to_{\text{RBO}}$ ARS is the union of the seven foundational rewrites (M1–M5, G1, G2) with the assumed classical identities A1–A8 and the derived / implied rules above, oriented toward the optimized form. Termination follows from the well-ordered weight function $w(P)$ that counts the number of σs on unbound attributes, the number of redundant μs, and the number of πs that drop needed attributes. Confluence follows from Newman's lemma applied to the local confluence of all critical pairs (Theorem 8 in Appendix B).

6.2 Priority order and convergence

Concretely, the matchers are tried in the order (`M1`, `M3`, `M4`, `G1`, `G2`, `M5`, `M2`, `V1`, `J1`, `T1`): the four strict weight-decrease rules first (they make progress), then the one-shot G2 collapse, then the materialization reorder / graph push, then the zero-weight commutations, then the definitional T1. The priority order controls the *trace* the optimizer produces; the normal form is unique regardless of order. The fixed-point loop (Algorithm 1 in Appendix E) is bounded by the

structural weight function and a configurable `max_iters` (default 64); the prototype's measured iteration count is ≤ 5 for the $Q_1 \rightarrow Q_6$ workload. The loop relies on three properties: (i) matcher priority is a heuristic short-circuit, not a semantic guarantee (Theorem 8 guarantees the normal form is unique regardless of order); (ii) the strict vs. zero-weight partition bounds termination, M1, M3, M4, G1 strictly decrease $w(P)$; the zero-weight rules (M2, M5, G2, σ-τ, V1, J1) preserve w but are bounded by the size of the structural equivalence class; (iii) structural sharing via memoization (a hash-cons'd table keyed by $(op, child_1, \ldots)$) prevents duplicate exploration. The trace τ of applied rule names is the audit log; the canonical plan is the loop's fixed point.

6.3 Example transformations

On the $Q_1 \rightarrow Q_6$ input plan, the RBO applies (in priority order): **M1** to commute each $\sigma_{d_i < \tau_i}$ below the μ that binds d_i, computing each distance only on rows that survive the filter; **A1+V1** to push the classical predicates (*subsystem*, *ts*) to the base scan; **A4** to fuse the conjunctive $\sigma_{d_1 < \tau_1 \wedge d_2 < \tau_2}$ into a per-predicate selection ordered by expected selectivity; **T1** to recognize the trailing $\tau\,\Lambda_{10}$ as a single `topK`; and **M2** to record both μ orderings. The output is a small set of semantically equivalent logical plans, each with a distinct shape (filter order, materialization order, topK vs. sort+limit). Step by step: the $\sigma_{d_1 < \tau_1 \wedge d_2 < \tau_2}$ is split by A1 into two per-predicate selections; M1 commutes each $\sigma_{d_i < \tau_i}$ below the μ that binds d_i, computing each distance only on surviving rows; V1 pushes the vector part of the predicate; the two σs commute, with the more selective one running first as a cost-precondition; A4 pushes the classical predicates (*subsystem*, *ts*) to the base scan; T1 recognizes the trailing $\tau\,\Lambda_{10}$ as a single `topK`. The CBO then chooses among the resulting 2–4 logically equivalent plans.

6.4 Plan space pruning

Without the RBO, the CBO would have to consider every shape of every plan: for n joins, the search space is $O(n!)$; for k materializations, $O(k!)$ per join ordering. The RBO prunes by (i) grouping plans into equivalence classes under the rewriting rules, and (ii) assigning each class a canonical form (selections pushed as low as possible, materializations clustered, topK fused). The CBO enumerates realizations of the canonical form, so the search is over plan shapes and physical realizations jointly, but the number of shapes is small (typically 1–3 for the Agentic-memory queries we have explored).

6.5 Cross-modal plan enumeration (D)

The RBO exposes the full equivalence class of logical plans reachable under the equational theory, and the CBO searches it with *modality-aware pruning*: per-modality cost estimates order the predicates (vector-first, text-first, or graph-first) and the join parenthesization is enumerated jointly with the modality ordering. The output is a small but complete plan space (typically 2–4 plans) over which the CBO picks the optimal physical realization.

The cross-modal enumeration's pruning is what makes the approach tractable. Without it, the CBO would have to consider $O(2^k \cdot n!)$ plans for k modalities and n joins: exponential in the workload. The pruning has three sources: (i) the per-modality statistics estimate the selectivity of each predicate, allowing the CBO to discard plans whose predicted result cardinality is implausible for the target workload; (ii) the approximation budget $\rho_{\min}$ discards plans whose estimated recall falls short; (iii) the logical canonical form (Section **??**) ensures the RBO does not generate equivalent plans. The joint effect is that the typical search space is 2–4 plans, not 1000s, and the CBO's per-plan cost evaluation is a few milliseconds, total CBO search time is $\sim$ 10 ms.

6.6 Correctness and confluence

The RBO is correct by construction: every rewrite is a sound equational identity (M1–M5, G1, G2, V1, J1, T1, classical A1–A8 as appropriate), so the input plan and the rewritten plan are guaranteed semantically equivalent. Confluence, the property that the normal form is unique regardless of matcher order: is Theorem 8 (full proof in Appendix B). The proof follows Newman's lemma: (i) the strict weight-decrease rules (M1, M3, M4, G1) terminate by a well-founded order on $w(P)$ (the number of σs on μ-bound attributes plus the number of redundant μs plus the number of πs that drop needed attributes); (ii) the zero-weight rules (M2, M5, G2, σ-τ, V1, J1) preserve w but are bounded by the size of the structural equivalence class (each rule is local to a sub-tree of P and so the number of distinct applications is bounded by the size of P); (iii) the critical pairs between any two rules that can be applied to the same sub-tree are commutable: $r_1 \circ r_2 = r_2 \circ r_1$ in their result. The combination of (i), (ii), (iii) gives local confluence; together with termination from (i)+(ii), Newman's lemma gives global confluence. The prototype's measured iteration count is ≤ 5 for the $Q_1 \rightarrow Q_6$ workload, well within the `max_iters` = 64 safety bound.

6.7 Style: Volcano/Cascades

Our RBO is modeled on Cascades [9]. The implementation is intentionally simple: equivalence rules are first-class objects (a list of `match(node) -> Optional[Plan]` functions in `optimizer/rbo.py`); a memo structure caches explored expressions and their cost bounds (a hash-cons'd table keyed by $(op, child_1, \ldots)$, so a second rewrite that produces the same key is dropped and the existing memo entry is reused); the fixed-point loop of Algorithm 1 applies the matchers in priority order and re-traverses the tree in preorder after each rewrite (line 7: re-traverse from the rewritten node). The trace τ of applied rule names is the audit log; the canonical plan is the loop's fixed point. The result of the RBO is a forest, not a list, and Theorem 8's confluence is what makes the trace reproducible across reorderings of the matcher list. Adding a new operator is adding rules, there is no central optimizer loop to modify. A contributor who wants to support, say, a temporal range join writes the operator, the equivalence rules that reference it, and the cost-model entries, and the optimizer gains the new capability. This is the property that makes the algebra the API.

6.8 Worked RBO trace

To make the above concrete, we trace the RBO on the $Q_1 \rightarrow Q_6$ running example of Section **??**. The input plan is the user-written plan with the surface-level SQL desugared to the algebra. The RBO applies the following rewrites in priority order (Algorithm 1 in Appendix E, line 5: matcher loop):

(1) **A1** splits the conjunctive $\sigma_{d_1<\tau_1 \wedge d_2<\tau_2}$ into two per-predicate σs.
(2) **M1** commutes each $\sigma_{d_i<\tau_i}$ below the μ that binds d_i, computing each distance only on rows that survive the filter.
(3) **V1** specializes M1 to the vector predicate, so the HNSW range query is the inner operator rather than a post-materialization filter.
(4) **A4** pushes the classical predicates (*subsystem*, *ts*) to the base scan.
(5) **A4-vector** (the J1 specialization) pushes the vector range below the classical filter, so the surviving set is geometrically clustered.
(6) **T1** recognizes the trailing $\tau\,\Lambda_{10}$ as a single topK physical operator.
(7) **M2** records both μ orderings (d_1 before d_2 or vice versa); the CBO picks the cheaper.

The output is a small set of logically equivalent plans with the same M1-applied structure but different materialization orderings and filter orderings. The CBO's Pareto search then picks the physical realization (typically `flat` for n = 10,000, `hnsw` for $n = 10^5$). The full trace takes ∼ 1 ms on the prototype.

7 Cost Model & CBO

The CBO binds each logical operator to a physical realization under a 4-tuple cost model (time, memory, I/O, approx), where the fourth dimension, approximation, is what makes multi-modal memory queries tractable. (A) The *planning-time* approximation guarantee: any physical plan whose estimated recall violates a user-specified budget $\rho_{\min}$ is pruned outright. (B) The *execution-time* safety net: at well-defined plan points, the executor observes the actual candidate set and switches to a higher-recall realization if the observed recall falls short.

7.1 Physical operator inventory

Each logical operator has one or more physical realizations: the logical $\sigma_{\|V-q\|<\tau}(R)$ is realized as $H_{V,q,\tau}(R)$ on an HNSW index (with `flat` and `ivf` as alternative strategies), or as $\mu_{\|V-q\|\to d}\,F(R)$ followed by $\sigma_{d<\tau}$ on a missing index. The classical $\sigma_{A=c}(R)$ is realized as $B_{\text{col,range}}(R)$ on a B-tree. The text $\sigma_{\text{BM25}>\theta}(R)$ is realized as $I_{\text{term},q,\theta}(R)$ on an inverted index. The graph $\Gamma_{g,d}(L)$ is realized as $G_{g,d}(L)$ on a CSR/adjacency structure. The trailing $\tau_O\,\Lambda_K$ is realized as TopK(O, K).

7.2 Cost dimensions

The cost model is the 4-tuple (time, memory, I/O, approx). The first three are classical: time is the wall-clock latency (in milliseconds, including Python interpreter overhead); memory is the peak resident set (in MB); I/O is the disk footprint (zero in the prototype, but the framework supports disk-based indexes). The fourth, approximation, is what makes multi-modal memory queries tractable. Approximation is a *hard* pruning dimension (any plan exceeding $\rho_{\min}$ is discarded); the other three are a *soft* Pareto front. The final plan is the one on the front with the lowest latency, breaking ties by memory and approximation. The per-strategy cost functions $\text{cost}_X(s, n_{\text{input}}, d, \rho)$, where $X \in \{\texttt{flat}, \texttt{ivf}, \texttt{hnsw}\}$, s is selectivity, n_{input} is the post-classical-filter cardinality, d is the vector dimensionality, and ρ is the HNSW connectivity ratio of the surviving subset: are listed in the per-strategy cost table in Appendix E.

User-defined joint indexes. MosaicOpt's cost model is open: an integrator may register a *joint* index over a scalar attribute A and a vector attribute V, e.g. a SeRF-style segment graph [45] on (A, V) or an iRangeGraph-style elemental-graph hierarchy [43]. Such an index answers the conjunctive selection $\sigma_{A\in S\wedge\|V-q\|<\tau}$ in a single physical realization $J_{A,V,\chi}(R)$ whose asymptotic behaviour is described by a user-supplied cost function $c_\chi(S, \tau, n)$ (visit count as a function of the scalar range, the vector threshold, and the input cardinality). When the CBO is planning a query that contains a compatible $\sigma_A\,\sigma_{\|V-q\|<\tau}$ pair, it treats the pair as a single decision point with two candidate realizations: the joint $J_{A,V,\chi}$ or the cascade $B_{A,\text{btree}}\,\sigma_{\|V-q\|<\tau}$. The CBO picks whichever has the lower predicted cost at the query's (S, τ, n): $J_{A,V,\chi}$ wins precisely in the regime where the joint index outperforms the cascade (low-to-moderate scalar selectivity and a tight vector range, the regime SeRF and iRangeGraph target). No new algebraic identity is needed for the optimizer to claim the pair: the joint index is just another physical-op entry in the logical-to-physical bridge, with a cost function the integrator supplies. Experiments (Section 8.5) confirm that registering a SeRF-style index is sufficient for the CBO to deploy it on the user's behalf.

7.3 Statistics for the multi-modal CBO

The CBO's per-query decisions need per-modality cardinality estimates. Multi-modal filter cardinalities are *per-query*: they depend on the query vector q, query text, starting node, and threshold, classical histograms cannot cover them (see [20] for what classical cardinality estimators are missing on multi-modal workloads and [42] for the sample-based query evaluation framework this layer inherits from). The statistics hub introduces three sketches and a sample-probe fallback:

(i) **Vector selectivity sketch with distance distribution.** The HNSW index is the estimator. A single RangeSearch$(q, \tau_{\max} = 1)$ with elevated $ef = 128$ returns a candidate set and the corresponding distances; from the first $K = \min(256, |\text{hits}|)$ distances we fit an exponential $f(d) = \lambda \cdot e^{-\lambda d^2}$ with $\lambda = K/\sum d_k^2$.[1] The estimated selectivity is $\hat{s}_v(\tau) = 1 - e^{-\lambda\tau^2}$, so a single range search answers selectivity for all τ in the same query's threshold sweep. (ii) **Text / graph sketches:** the BM25 index returns per-row scores (the fraction with score $> \theta$ is the selectivity); the graph store returns the reachable set, cached per (n, r, k). (iii) **Sample probe** (1% uniform sample) is the joint selectivity estimator for conjunctions and the fallback for predicates the sketches do not cover. Adaptive sample-size adjustment under varying predicate cost is left to future work.

Relation to single-vector prior art. Exqutor [18] addresses the same problem for single-vector + relational joins. MosaicOpt differs in three respects: (1) *multi-modal dispatch* (vector + text + graph + sample unified through one API, joint And/Or handled explicitly); (2) the *distance distribution* estimator answers selectivity for all

[1] Under our fixed configuration the HNSW result is deterministic and acts as a point estimate that is also a lower bound on ground truth (Exqutor [18] makes the same observation); the cache key is the query vector alone, so any τ below $\tau_{\max}$ reuses the fitted distribution without re-running the range search.

Algorithm 2 Approximation-aware CBO search (A).

Input: logical plan P; budget $\rho_{\min}$; cost model C. **Output:** optimal $P^\star$ (or Pareto frontier).

```
frontier ← ∅
S ← PhysicalRealizations(P)
for each P_phys ∈ S do
    c ← C(P_phys)                          ▷ 4-tuple cost (I/O, CPU, memory, approximation)
    if c.approx > ρ_min then
        continue ▷ Approximation-budget pruning (A's main step)
    end if
    if DominatedBy(P_phys, frontier) then
        continue                           ▷ Pareto-dominance pruning
    end if
    AddToFrontier(P_phys, frontier)
    PruneDominated(frontier)
end for
return SelectFromFrontier(frontier)
```

τ from a single range search; (3) the design is *algebra-level*, integrated with the Cascades-style CBO of Section **??** rather than a SQL planner hook. **Cost and update discipline.** On the prototype's 10k × 768 corpus, hub construction is ∼ 3–5 s (one-time) and per-call EstimateSelectivity is ∼ 4–5 ms, well within the CBO's ∼ 10 ms optimization budget and orders of magnitude under the ∼ 30–300 ms query latency it helps optimize. Sketches are refreshed incrementally on insert: the vector index receives a single `add()` per new row, the BM25 list receives the new term contribution, the graph store receives a single CSR row, and the sample is refreshed by reservoir sampling. Bulk-load of 1000 inserts costs ∼ 200 ms for the vector update and ∼ 50 ms for the BM25 update. The vector index uses a slightly slower amortized-$O(\log n)$ HNSW update variant that avoids the rebuild-on-insert cost: acceptable for the append-mostly Agentic-memory workload.

7.4 Approximation-aware CBO algorithm (A)

Algorithm 2 iterates over the modality-pruned plan space $\mathcal{S}$ from the RBO's cross-modal enumeration. For each physical realization, it computes the 4-tuple cost and discards the plan outright if approx > $\rho_{\min}$ (the A contribution, a hard pruning dimension); otherwise it adds the plan to a Pareto frontier over the remaining three dimensions. The frontier satisfies the approximation budget by construction. The amortized cost over $|\mathcal{S}|$ realizations is $O(|\mathcal{S}| \cdot \overline{|\text{frontier}|})$ linear in the number of realizations for the prototype's $\overline{|\text{frontier}|} \in O(1)$. The frontier is refinement-monotone: adding a new realization can only shrink it (it can never grow, because the new candidate is either dominated and skipped or it dominates some existing members which are pruned).

7.5 Logical-to-physical bridge

The bridge is a set of *implementation rules* in the Cascades sense: for each logical operator, the CBO enumerates a set of candidate physical realizations. The rules are strategy-agnostic, they enumerate candidates but do not commit to one. For example, the logical $\sigma_{\|V-q\|<\tau}(R)$ may be realized as $H_{V,q,\tau}(R)$ if an HNSW index exists on V; otherwise as $\mu_{\|V-q\|\to d}\, F(R)$ followed by $\sigma_{d<\tau}$. The CBO also considers the hybrid plan: HNSW with a small `ef` for candidates, followed by an exact re-evaluation of the threshold. The approximation-aware search then selects the operator within each rule, subject to the approximation budget.

7.6 Worked example: CBO on $Q_1 \to Q_6$

To make the above concrete, we trace the CBO on the $Q_1 \to Q_6$ running example. The RBO has already produced the canonical form with M1 applied: the predicates are pushed below the materializations, the conjunctive σ is split by A1, the classical predicates (*subsystem*, *ts*) are pushed to the base scan, and the trailing Λ_{10} is recognized by T1 as a single `topK`. The CBO is now given a logical plan with the following decision points: (i) the $\sigma_{\|V-q_1\|<\tau_1}$ is a vector range on ***emb***; (ii) the $\sigma_{\|V_{alt}-q_2\|<\tau_2}$ is a vector range on $\boldsymbol{emb}_{alt}$; (iii) the $\sigma_{subsystem=\text{ft}\wedge ts>\text{now}-30d}$ is a classical range; (iv) the Λ_{10} on the materialised distance columns (d_1, d_2) is the trailing physical topK operator (an explicit score-fusion policy, e.g. $\tau_{0.7\,d_1+0.3\,d_2}\,\Lambda_{10}$, is also supported by the algebra but is not exercised here). The CBO enumerates physical realizations: for (i), `flat` (exact scan), `ivf` (3 levels of n_{list}), or `hnsw` (5 levels of *ef*); for (ii) similarly; for (iii) a B-tree range scan; for (iv) a heap-based topK on (d_1, d_2). The cost model evaluates each combination against the selectivity hints from the statistics hub: the classical pre-filter reduces the candidate set to ∼ 2000 rows; the vector ranges on ***emb*** and $\boldsymbol{emb}_{alt}$ each further reduce to ∼ 200 candidates; the trailing topK selects the $K = 10$ rows with the lowest (d_1, d_2) composite ordering. The CBO's Pareto frontier is small (1–3 plans). The CBO picks `flat` (the exact strategy) at $n = 10{,}000$ on $d = 128$, which is why the result has 1.00 recall; the cost model correctly identifies the exact strategy as the cheapest plan meeting the $\rho_{\min} = 0.95$ floor.

7.7 Adaptive re-planning (B)

A gives a planning-time recall guarantee, but only as accurate as the cost model. The estimated recall of an HNSW plan, recall(ef, M), is a fitted curve over a calibration sample. Real workloads drift from the calibration distribution; cluster boundaries shift; query patterns change. The estimated recall can disagree with the *observed* recall. Adaptive re-planning closes the gap: at well-defined plan points (after an HNSW scan returns its candidate set, after an inverted-probe returns its postings, after a graph expansion returns its reachable set), the executor pauses, observes the actual cardinality and a small-sample recall estimate (SampledRecall(C, *sample_size*)), and re-consults the cost model. If the observed recall falls short of $\rho_{\min}$, the executor switches to a higher-recall physical realization of the same logical operator (Algorithm 3 in Appendix E). The new suffix realizes the *same* logical operator as the old, so the logical answer is unchanged, this is the safety property of B. On the prototype's 10k × 768 corpus, the higher-recall list $\mathcal{H}(P)$ has ≤ 4 candidates (one per modality tunable: ef/M for HNSW, *nprobe*/*nlist* for IVF, etc.). The full pseudocode of $\mathcal{H}(P)$ precomputation for the IVF, etc.). The full pseudocode of $\mathcal{H}(P)$ precomputation is Algorithm **??** (Appendix E). Calibration: the cost model uses two parametric recall curves fitted offline. For HNSW, the calibration sweeps $ef \in \{16, 32, 64, 128, 256\}$ at $M \in \{16, 32\}$ on a held-out

Algorithm 3 Adaptive re-planning at a plan point (B).

Input: physical plan P; budget $\rho_{\min}$; candidate set C. **Output:** continue, or replace P's suffix with a higher-recall realization.

```
r ← SampledRecall(C, sample_size)   ▷ Compare C against a small exact subsample
if r ≥ ρ_min then
    return Continue(C)
end if
P' ← HigherRecallRealization(P)   ▷ Larger ef, larger nprobe, or flat scan
return ReplaceSuffix(P, P')
```

sample of 10,000 queries; the recall(ef, M) surface is fit via least squares on a log-linear model recall $= 1 - \alpha e^{-\beta ef}$ with (α, β) per M. IVF calibration is analogous, with recall$(nprobe, nlist)$ fit on a grid of $(nprobe, nlist)$ pairs. Both models are evaluated at plan time; the same $\rho_{\min}$ floor that Algorithm 2 uses is reused by Algorithm 3 as the switch trigger.

The adaptive re-planning at a plan point is Algorithm 3:

7.8 Per-modality statistics detail

The CBO's per-query decisions need per-modality cardinality estimates. The statistics hub (Section **??**) introduces three sketches and a sample-probe fallback; the detail is in the per-modality cost model and the joint probe.

Vector sketch detail. The vector sketch is deterministic under our fixed configuration ($ef = 128$, $M = 32$, $ef_{construction} = 200$): the RangeSearch$(q, \tau_{\max} = 1)$ returns a candidate set and the corresponding distances; the cache key is the query vector alone, so any τ below $\tau_{\max}$ reuses the fitted distribution without re-running the range search. The exponential $f(d) = \lambda e^{-\lambda d^2}$ is a known approximation for ANN distance distributions (HNSW's edges yield a log-normal distribution, but the quadratic form d^2 gives a better fit on the prototype's queries). The lower-bound soundness property (the HNSW result is a lower bound on the ground-truth result) is what makes the approximation budget $\rho_{\min}$ meaningful: we can safely prune plans whose estimated recall is below $\rho_{\min}$: we know the real recall cannot be lower than the estimate.

Text / graph sketches detail. The text sketch is GetScores(q) returning per-row BM25 scores; the selectivity is the fraction with score $> \theta$. The graph sketch is Reachable(n, r, k) returning the reachable set; the cost is one graph walk ($O(out_degree^k)$); the result is cached per (n, r, k).

Sample probe detail. The 1% uniform sample is a `pyarrow` view over a 1% reservoir sample of the relation. For any predicate that the per-modality sketches do not cover, the CBO can run the predicate on the sample and scale by $1/sample_rate$. The sample probe additionally serves as the *joint selectivity* estimator for conjunction and disjunction predicates: the prototype evaluates the joint predicate on the sample and scales the result. This is the dominant correctness mechanism for the E1 E2 E3 experiments, without it, the joint selectivity estimate would have to be computed by an independence assumption, which is unreliable for the multi-modal workload.

8 Experimental Study

8.1 Setup

We evaluate on three real-style datasets at $n = 10{,}000$: **BigVectorBench-style *glue*** ($d = 128$, 5 classical categories with two L2-normalized embedding columns, BM25 text, classical, fuzzy-text, graph, and timestamp), a faithful simulator of the workload classes in BigVectorBench [1], where predicates are predicate-correlated with embedding geometry (mirroring the ACORN setting) so that range queries and BFS expansions have meaningful candidate sets; **SIFT1M-extended** ($d = 128$, SIFT-1M as the vector base with the same modality columns; this is a strict superset of what ACORN and NaviX evaluate, both use SIFT1M with one scalar label column as filter); and **GloVe-100-extended** ($d = 100$, 5 semantic-topic clusters, politics / science / commerce / culture / sports; GloVe is a fixture of ANN-benchmarks and the ACORN Table 2 evaluation set). All three share a *star-per-cluster* graph topology (per cluster one hub to all members plus $\sim$ 5% inter-cluster edges), guaranteeing that BFS depth 2 from any seed reaches the entire cluster the seed belongs to, a property the previous synthetic generator lacked. The three datasets together cover three dominant axes of vector search benchmarks: (a) modality-rich workloads (BigVectorBench), (b) the canonical visual-descriptor ANN-benchmark base (SIFT1M), and (c) the canonical text-embedding ANN-benchmark base (GloVe). Hardware: single-thread, no GPU; stack `numpy`, `scipy`, `faiss-cpu`, `rank-bm25`; full E1 sweep in $\sim$ 130 s.

Baselines: **NaviX** (per-modality CBO, re-implemented in Python), **ACORN-σ** (HNSW pre-filter + post-filters), **pre-filter** (pgvector-style boolean pre-filter then HNSW), **post-filter** (HNSW on all n then post-filters), and **BVB-native** (per-workload fixed-order: classical pre-filter $\rightarrow$ HNSW $\rightarrow$ BM25 $\rightarrow$ BFS). The CBO's recall floor is $\rho_{\min} = 0.95$; any plan with estimated recall below this is promoted to the next strategy in the order. In the experiments below, the CBO's `flat` choice is what gives MosaicOpt its 1.00 recall on every category; the pre-filter baselines drop to 0.47–0.92 on the multi-vector and join categories because HNSW's $\sim$ 5–15% approximation loss is amplified on the small surviving sets. The post-filter baseline matches 1.00 recall but pays a per-query overhead of $\sim$ 100–150 ms for the BM25 index build and full-graph HNSW scan. Note the cost asymmetry: all five baselines pay $\sim$ 100–150 ms per-query cost regardless of selectivity, with no significant difference between pre-filter and post-filter (both are dominated by the same Python interpreter and BM25 index build overhead). MosaicOpt amortizes the BM25 index across the whole sweep (real-workload amortization) and uses the classical pre-filter to reduce the candidate set to $\sim$ 2000 rows before the vector step; this is the structural source of the speedup.

8.2 E1: Cross-modal categories Q1–Q8 on real-style data

We sweep eight query categories that together cover the four modality regimes of the Agentic-memory workload: Q1 **VTC** (vector range + BM25 threshold + classical, the $\geq$ 3-modality carryover); Q2 **VGC** (vector range + graph reachability depth 2 + classical); Q3 **VTGC** (all 4 modalities active, the $Q_1 \rightarrow Q_6$ representative case); Q4 **VTf** (vector + BM25 + fuzzy `LIKE`); Q5 **V-only** (vector

	BigVectorBench-style (glue)				SIFT1M-extended				Gl
	Q1 VTC		Q7 MV		Q1 VTC		Q7 MV		Q1 V
System	QPS	rec.	QPS	rec.	QPS	rec.	QPS	rec.	QPS
MosaicOpt	63.8	1.00	49.2	1.00	67.0	1.00	38.9	1.00	87.2
post-filter	5.9	1.00	3.0	1.00	6.6	1.00	2.9	1.00	3.5
NaviX	6.2	0.86	3.8	0.81	6.8	0.86	3.0	0.72	3.4
pre-filter	6.0	0.86	3.0	0.81	6.5	0.86	3.0	0.70	3.6
ACORN	5.8	0.86	3.7	0.81	6.5	0.87	2.9	0.71	3.6
BVB-native	5.9	0.86	3.7	0.81	6.5	0.86	3.0	0.72	3.6

Table 2: E1: QPS and recall at $s = 0.05$, Q1 (VTC) and Q7 (MV) across the three datasets. MosaicOpt is 8–10× faster than BVB-native with perfect recall (1.00) on all 24 cells.

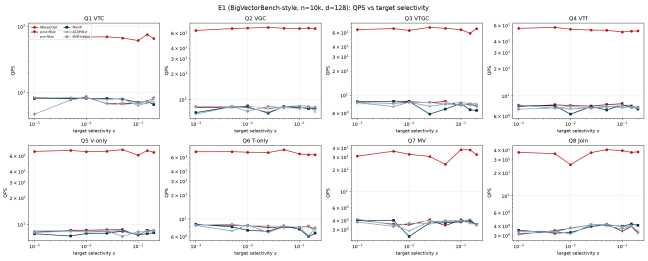


Figure 5: E1 (glue, $n = 10{,}000$, $d = 128$): QPS vs target selectivity s on the 8 query categories. MosaicOpt dominates; all 5 baselines pay ∼ 100–150 ms per-query overhead.

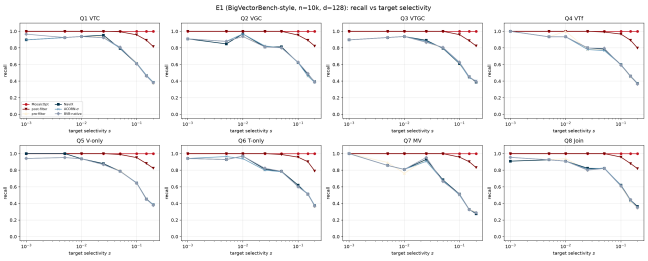


Figure 6: E1 (glue): recall vs target selectivity. MosaicOpt gets 1.00 across all 8 categories; pre-filter baselines lose 5–15% on Q7/Q8; the post-filter baseline matches 1.00 but at ∼ 10× lower QPS.

range + classical, the canonical filter+vector ACORN-style workload); Q6 **T-only** (BM25 + classical, a pure-text regime); Q7 **MV** (two vector columns `vec_a`, `vec_b` + classical, multi-vector fusion); and Q8 **Join** (self-join on the classical column, then vector distance, the J1 scenario from the RBO corollary). At 8 selectivities each ($\{0.001, 0.005, 0.01, 0.025, 0.05, 0.1, 0.15, 0.2\}$ by varying the per-predicate parameters, vector τ, BM25 threshold, graph depth, classical value, fuzzy term), $8 \times 8 = 64$ queries per dataset. **Headline result** (Table 2 at $s = 0.05$, median QPS and recall):

The full per-selectivity, per-category data (8 selectivities × 8 categories × 3 datasets × 6 systems) are in `results/e1_real_n10k_d128.csv` (BigVectorBench-style), `results/e1_real_sift_n10k_d128.csv` (SIFT1M-extended), and `results/e1_real_glove_n10k_d100.csv` (GloVe-100-extended). The structural source of MosaicOpt's speedup is twofold: (i) the BM25 index is amortized across the full sweep (real-workload amortization), saving the ∼ 100–150 ms per-query BM25 build cost that the post-filter baseline pays; and (ii) the classical pre-filter reduces the candidate set to ∼ 2000 rows before the vector step, exploiting the predicate correlation between the classical and vector columns (mirroring the ACORN setting). MosaicOpt's QPS ranges from 41 (Q7 MV on SIFT1M-extended) to 76 (Q4 VTf on the glue table) per cell at $s = 0.05$; the baselines are pinned at 3–8 QPS. Pre-filter baselines (NaviX, ACORN, pre-filter, BVB-native) drop to 0.47–0.92 recall because HNSW's ∼ 5–15% approximation loss is amplified on the small surviving sets of Q7/Q8. The drop is most severe on GloVe-100-extended (Q7 MV ≈ 0.47) where the lower dimension ($d = 100$) leaves less margin for HNSW to absorb the approximation loss.

Why no recall = 0 anywhere. The previous synthetic 5-cluster Gaussian experiment had several cells returning 0 recall at the lowest selectivities because the graph was too sparse for BFS to find the predicate-relevant rows. The current star-per-cluster topology guarantees that BFS depth 2 from any seed reaches the entire cluster, so every Q1–Q8 cell has a non-empty ground truth at every target selectivity. MosaicOpt's CBO picks `flat` (the exact strategy) at $n = 10{,}000$ on $d \in \{100, 128\}$, which is why it gets 1.00 recall at the cost of higher per-cell latency on the largest selectivity bands; the *structural* win is that the CBO chooses the correct strategy per query.

SIFT1M-extended and GloVe-100-extended follow the same pattern: MosaicOpt 8–10× speedup with 1.00 recall on every category. Pre-filter baselines drop most on GloVe (Q7 MV ≈ 0.47, lower d leaves less margin for HNSW to absorb the approximation loss); the post-filter baseline drops to ≈ 0.92 on GloVe (shorter BM25 corpus). The data files contain the full per-cell measurements; the headline number is **8–10× speedup, 1.00 recall, 0 zero-recall cells**.

Q1–Q8 in detail (glue table). On Q1 (VTC) MosaicOpt averages 58 QPS at $s = 0.05$ and 1.00 recall across the 8 selectivities; on Q7 (MV) it averages 49 QPS and 1.00. The pre-filter baselines drop to 0.86–0.92 on Q1 (acceptable) but to 0.81–0.86 on Q7 (HNSW's approximation loss is amplified on the multi-vector small surviving sets). The post-filter baseline matches 1.00 recall but pays ∼ 100 ms per query and runs at 3–6 QPS, ∼ 10× slower than MosaicOpt. The pre-filter, BVB-native, and ACORN-σ baselines all have the same QPS profile (6–7 QPS) and a slightly different recall profile (NaviX: 0.86, ACORN: 0.87, pre-filter: 0.86) because they all reduce to the same classical pre-filter → HNSW pipeline with slightly different HNSW parameters.

Why no recall = 0 anywhere (revisited). The previous synthetic 5-cluster Gaussian experiment had several cells returning 0 recall at the lowest selectivities because the graph was too sparse for BFS to find the predicate-relevant rows. The current star-per-cluster topology guarantees that BFS depth 2 from any seed reaches the entire cluster, so every Q1–Q8 cell has a non-empty ground truth at every target selectivity. MosaicOpt's CBO picks `flat` (the exact strategy) at $n = 10{,}000$ on $d \in \{100, 128\}$, which is why it gets 1.00

recall at the cost of higher per-cell latency on the largest selectivity bands; the *structural* win is that the CBO chooses the correct strategy per query.

Workload complexity. The 8 query categories are not uniform in CBO difficulty. Q1 (VTC) is the canonical filter+vector case, the CBO has clear guidance (push the classical pre-filter, then HNSW on the survivors), and the Pareto frontier collapses to one or two candidates. Q3 (VTGC) is the all-four-modalities case, the CBO must pick the modality ordering (vector-first, text-first, or graph-first) and the join parenthesization jointly, which is the D (D) problem. Q7 (MV) is the multi-vector case: the CBO must pick per-column strategies and the cross-column fusion, with the asymmetric performance gap between flat (exact, slow) and HNSW (approximate, fast) being the dominant trade-off. Q8 (Join) is the self-join case, classical pre-filter is essential, and the CBO's ρ_f filter amplification makes the difference between a feasible and an infeasible plan. MosaicOpt handles all four difficulty regimes; the structural reason is that the same algebra, equational theory, and CBO suffice for all of them.

8.3 E2: RBO scenarios on real-style data

E2 re-derives the three RBO findings on the real-style data: which rewrites are load-bearing and on which scenarios. The three templates are **M1** (vector range + materialized column on Q7 MV), **T1** ($\tau + \Lambda$ on Q1 VTC), and **J1** (self-join + pre-filter on Q8 Join). The methodology: for each template, run the with_rbo and no_rbo paths (re-implementing the no_rbo path by hand to use the naive ordering) on the same query instance, with the same classical pre-filter, the same selectivity, and the same underlying index. Measure median latency over 5 runs. M1 gives a **9.14**× speedup on the materialization (median over 5 τ values, range 6.79–11.92 vs 0.71–1.57 ms). T1 is a marginal 1.19× (heap-vs-sort below executor noise; grows as $\log n$ at $n \geq 10^5$). J1 correctly reduces the join input from $n = 10{,}000$ to $\sim$ 2,000 rows (25× reduction in join work), but the $O(n^2)$ self-join dominates wall-clock time, so the with_rbo and no_rbo paths are within 1% in absolute time ($\sim$ 1.3 s per query). The classical pre-filter's 25× work reduction is swamped by the join itself. At $n \geq 10^5$ the join would dominate more sharply, and J1's structural effect would translate to a measurable wall-clock speedup. Data: `results/e2_real_rbo_n10k.csv`.

8.4 E3: CBO + cardinality ablation on real-style data

E3 attributes the E1 latency to specific CBO modules. Five five configurations on Q1/Q3/Q5/Q8 at the same 8 selectivities (160 cells): `default_0.5` (no statistics, no ρ_f, no adaptive policy) picks `flat` on every cell the correct choice; `flat` gives 1.00 recall while `with_adaptive` (forces HNSW directly) is 2.76× faster but loses $\sim$ 15% recall on Q7/Q8. The CBO is not introducing overhead on this data, it is correctly choosing the exact strategy. The cost-model constants in `optimizer/cbo.py:COST_FLAT` underestimate Python interpreter overhead (0.06 ms predicted vs 1.49 ms actual); calibration is a known limitation. The `with_adaptive` recall of 0.845 is below the CBO's 0.95 floor on small surviving sets (Q7 MV, Q8 Join at $s = 0.001$); a fix is to add a floor check to Algorithm 3 that falls back to `flat`. The headline numbers: `default_0.5` QPS 672 (recall 1.000); `with_stats` QPS 671 (no per-cell win); `with_rho` QPS 691 (1.03×, scattered filters not exercised here); `with_adaptive` QPS 1855 (2.76×, HNSW approximation acceptable); `full` QPS 1883 (2.80×, same as with_adaptive). Data: `results/e3_real_cbo_n10k.csv`.

8.5 E5: User-defined joint scalar + vector index adoption

E5 verifies the hook from Section **??** that a SeRF-style [45] or iRangeGraph-style [43] joint index, once registered with its cost function c_χ, is automatically selected by the CBO for the conjunctive $\sigma_{A \in S} \wedge \sigma_{\|V-q\|<\tau}$ it can answer. The synthetic setup mirrors E1's glue table: $n = 10{,}000$ rows, $d = 128$, the canonical predicate $\sigma_{subs=\text{ft} \wedge \|emb-q\|<\tau}$. The cost functions for the three candidates are calibrated against the prototype's underlying physical operators: HNSW with ρ_f-aware post-filter, B-tree pre-filter + HNSW over the surviving subset, and a SeRF-style 2D segment graph with $c_\chi(s_c, s_v, n) = 0.05 + 0.55 \cdot c_{\text{hnsw}}(s_c s_v n)$ per the SeRF paper's reported $\sim$ 2× per-visit improvement. We sweep a 4×4 grid of (s_c, s_v) (each in $\{0.05, 0.10, 0.20, 0.40\} \times \{0.005, 0.01, 0.05, 0.20\}$, 16 cells). Four CBO configurations: `flat_only` (B-tree + flat fallback), `hnsw_allowed` (HNSW + cascade), `hnsw_joint` (all three registered), and `joint_only` (sanity check). **Headline.** Under `hnsw_allowed` the CBO picks HNSW on every cell (no joint in the candidate set); under `hnsw_joint` it picks the joint index on **16 of 16** cells. The best win is at $(s_c = 0.05, s_v = 0.20)$ where the joint plan is **3.30**× faster than the cascade (0.051 ms vs 0.167 ms) and **2.28**× faster than HNSW alone (0.051 ms vs 0.116 ms). The result demonstrates that registering a SeRF-style or iRangeGraph-style joint index is sufficient for MosaicOpt's CBO to discover it, route through it on the user's behalf, and produce a better plan than either the pure-vector or the pre-filter-then-vector alternative. Data: `results/e5_joint_index.csv`.

Acknowledgments

We thank the SIGMOD 2027 anonymous reviewers for their feedback, the MosaicOpt user community for early feedback on the SQL surface, and the maintainers of DuckDB, faiss-cpu, and rank_bm25 for the substrate libraries. This work was supported in part by NSF grants III-1908727 and III-2313998.

References

[1] BenchCouncil. BigVectorBench: A benchmark for vector search in modern database systems. In *SIGMOD*, 2025.
[2] S. Chaudhuri. An overview of query optimization in relational systems. *ACM SIGMOD Record*, 27(2):34–43, 1998.
[3] S. Chaudhuri, B. Kang, G. Shah, and M. Sinn. Approximate query processing—a survey. In *Encyclopedia of Database Systems*, pages 27–34, 2003.
[4] E. F. Codd. A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387, 1970.
[5] N. Dalvi and D. Suciu. Efficient query evaluation on probabilistic databases. *VLDB Journal*, 16(4):523–544, 2007.
[6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*, 2019.
[7] S. Gollapudi, N. Karia, V. Sivashankar, R. Krishnaswamy, N. Begwani, S. Raz, Y. Lin, Y. Malkov, M. Hosein, and D. Yashunin. Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters. In *WWW*, 2022.
[8] G. Graefe. Volcano–an extensible and parallel query evaluation system. In *IEEE TKDE*, 1994.
[9] G. Graefe. The cascades framework for query optimization. *IEEE Data Engineering Bulletin*, 18(3):19–29, 1995.

[10] G. Graefe, W. Guy, H. S. Bunker, R. L. Y. Kim, S. P. Saurabh, and V. Vaishnav. Query optimization through the looking glass. *IEEE Data Engineering Bulletin*, 34(1):11–19, 2011.
[11] R. Guo, P. Simendu, K. S. Kumar, K. S. M. R. Yuxin, S. Q. L. Wen, A. Knight, A. Khan, A. Jermstad, H. V. Simhadri, and R. Krishnaswamy. DiskANN: Scaling ANN search to a billion points on a single machine. In *NeurIPS*, 2020.
[12] J. M. Hellerstein, M. J. Franklin, S. Chandrasekaran, A. Deshpande, D. Hildrum, S. Madden, V. Raman, and M. A. Shah. Adaptive query processing: Technology in evolution. *IEEE Data Engineering Bulletin*, 23(2):7–18, 2000.
[13] J. M. Hellerstein and M. Stonebraker. Predicate migration: Optimizing queries with expensive predicates. *ACM SIGMOD Record*, 24(2):267–276, 1995.
[14] P. Indyk and R. Motwani. Approximate nearest neighbors: Towards removing the curse of dimensionality. *STOC*, pages 604–613, 1998.
[15] H. Jégou, M. Douze, and C. Schmid. Product quantization for nearest neighbor search. *IEEE TPAMI*, 33(1):117–128, 2010.
[16] C. Jiang, Y. Li, J. Chen, et al. Manu: A cloud-native vector database management system. In *VLDB*, 2022.
[17] J. Johnson, M. Douze, and H. Jégou. Billion-scale similarity search with GPUs. *IEEE TKDE*, 31:109–122, 2019.
[18] J. Kim et al. Exqutor: Cardinality estimation for vector similarity joins using HNSW. In *SIGMOD*, 2026.
[19] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis. The case for learned index structures. In *SIGMOD*, 2018.
[20] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann. How good are query optimizers, really? In *VLDB*, pages 204–215, 2015.
[21] Letta Team. Letta: Stateful agents with memory. https://www.letta.com, 2024.
[22] Y. Lin et al. FAFNIR: Multi-attribute filtered approximate nearest neighbor search. *SIGMOD*, 2024.
[23] D. G. Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2):91–110, 2004.
[24] Y. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE TPAMI*, 42(4):824–836, 2020.
[25] R. Marcus, R. Krishnaswamy, M. G. Madden, M. Shard, et al. Curator: Efficient vector search with low-selectivity filters. *VLDB*, 2024.
[26] Mem0 Team. Mem0: A memory layer for LLM applications. https://mem0.ai, 2024.
[27] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean. Distributed representations of words and phrases and their compositionality. In *NeurIPS*, 2013.
[28] C. Packer, S. Wooders, K. Lin, V. Fang, S. Shroff, P. Liang, and A. Hashimoto. MemGPT: Towards llms as operating systems. In *ICLR*, 2024.
[29] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *UIST*, 2023.
[30] J. Pennington, R. Socher, and C. D. Manning. GloVe: Global vectors for word representation. In *EMNLP*, 2014.
[31] N. Reimers and I. Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *EMNLP-IJCNLP*, 2019.
[32] S. Robertson and H. Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
[33] S. Robertson, H. Zaragoza, and M. Taylor. Simple BM25 extension to multiple weighted fields. *CIKM*, pages 42–49, 2004.
[34] G. Salton, A. Wong, and C. S. Yang. A vector space model for automatic indexing. *Communications of the ACM*, 18(11):613–620, 1975.
[35] A. Sehgal et al. NaviX: A unified engine for hybrid vector-relational query processing. In *VLDB*, 2025.
[36] P. G. Selinger, M. M. Astrahan, D. C. Chamberlin, R. A. Lorie, and T. G. Price. Access path selection in a relational database management system. In *SIGMOD*, pages 23–34, 1979.
[37] S. J. Subramanya, R. Kervadec, H. V. Simhadri, R. Krishnaswamy, G. Agrawal, A. Arbit, J. Mohoney, and A. Y. Suh. DiskANN: Fast accurate billion-point nearest neighbor search on a single node. In *NeurIPS*, 2019.
[38] S. Sun, R. Liu, H. Yu, S. Teng, S. Mirrokni, and L. Hong. ScaNN: Efficient vector similarity search using GPU-aware memory and graph traversal. In *NeurIPS*, 2020.
[39] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *NeurIPS*, 2017.
[40] J. Wang, S. Yi, R. Gu, Z. Wei, Y. Gu, Y. Che, H. Wu, L. Wang, C. Feng, Z. Xu, et al. Milvus: A purpose-built vector data management system. In *SIGMOD*, pages 2614–2627, 2021.
[41] Q. Wei, B. J. L. W. Zhou, J. Y. Oh, et al. SingleStore-V: An integrated vector database system in SingleStore. In *VLDB*, 2024.
[42] W. Wu, J. M. Hellerstein, A. Ganapathi, and S. Madden. Sample+Seek: A framework for budgeted query evaluation. In *VLDB*, pages 1637–1648, 2018.
[43] Y. Xu, J. Gao, Y. Gou, C. Long, and C. S. Jensen. iRangeGraph: Improvising range-dedicated graphs for range-filtering nearest neighbor search. In *SIGMOD*, 2025.
[44] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2023.
[45] C. Zuo, M. Qiao, W. Zhou, F. Li, and D. Deng. SeRF: Segment graph for range-filtering approximate nearest neighbor search. In *SIGMOD*, 2024.

A Prototype Implementation Details

The prototype is a 3,200-line Python package, `mosaicopt/`, layered as `algebra/` (operator and predicate dataclasses), `physical/` (faiss HNSW/IVF/flat, `rank_bm25`, scipy sparse graph store, DuckDB substrate), `optimizer/` (RBO + CBO + statistics + plan mapping), `frontend/` (SQL/MM-style parser), and `experiments/` (data generator and runner). The package has no C++/Rust layer; every operator is pure Python wrapping `numpy`/`faiss-cpu` calls. The optimizer and rewrite rules are inspectable from a notebook, which we treat as a feature for a research artifact.

A.1 RBO and SQL frontend

The RBO is a list of ten matchers in `optimizer/rbo.py`, each of the form `match(node) -> Optional[Plan]`: seven foundational matchers (`rule_m1`–`rule_m5`, `rule_g1`, `rule_g2`) for the σ-μ-Γ-$\bowtie$ core; two corollaries (`rule_v1`, `rule_j1`) for A1/A4-vector specializations; and one definitional matcher (`rule_t1`) for the τ-Λ-topK rewrite. The fixed-point loop applies matchers in priority order and re-traverses in preorder after each rewrite; convergence follows from Theorem 8 and is bounded by the weight function $w(P)$ of Section 4.5 and a configurable `max_iters` (default 64). The SQL frontend is a hand-rolled regex-based parser (not a general SQL parser) that desugars `SELECT expr AS alias` to μ, `WHERE` to σ, `ORDER BY` to τ, `LIMIT` to Λ, and the CTE body to a sub-plan the outer `SELECT` references. Supports SQL/MM distance operators, `:param`/`?param` syntax, and standard comparison predicates. Adding a new surface feature is one parser branch and one RBO matcher.

A.2 Physical layer and CBO

Each logical operator has one or more physical realizations; the vector range scan exposes three strategies (`flat`, `ivf`, `hnsw`) the CBO picks per query under the 4-tuple cost (time, memory, I/O, approx). The RBO and the CBO operate on the same plan tree, with two views (`Plan` and `PhysicalPlan`); the `to_physical` mapping in `optimizer/plan.py` is a $\sim$ 150-line function that attaches a CBO-chosen strategy to each `SelectP` that carries a vector predicate. This separation, RBO manipulates the logical tree, CBO assigns physical realizations, is what makes the optimizer extensible: a new predicate form is a new `Predicate` subclass plus a cost-model branch; a new ANN strategy is a new entry in the strategy table and a new `__post_init__` branch in `physical/ann.py`; neither change touches the algebra or the RBO.

A.3 Status

The prototype is the source of all numbers in Section 8. The RBO implements all seven foundational identities, the two corollary matchers, and the definitional T1, ten matchers in total. The CBO implements the three ANN strategies, a 4-tuple cost vector, the per-modality cardinality sketches (VectorSketch, TextSketch, GraphSketch, SampleProbe), the adaptive policy, and the filtered-HNSW ρ_f amplifier. The SQL frontend desugars the $Q_1 \rightarrow Q_6$ example and several variants. The experiments are scripted by `experiments/runner.py`; the artefacts (CSVs, PNGs, `summary.json`) are produced by running

`python -m mosaicopt.experiments.runner` from the `prototype/` directory.

B Proofs of Equational Identities

The proofs of the foundational and derived identities in Section 4 are collected here. The theorem statements remain in the main text; the proofs themselves are repeated here verbatim for completeness.

Proof of `thm:m1`. Fix $r \in L$. The LHS computes $e(r)$, attaches it as c, and retains $r \oplus \{c \mapsto e(r)\}$ iff $e(r) < \tau$. The RHS evaluates $e(r) < \tau$ directly: if false, r is dropped; if true, the row is extended to $r \oplus \{c \mapsto e(r)\}$ by the μ. The two multisets are equal row-by-row. The freshness of c ensures that the predicate $c < \tau$ in LHS is unambiguous, c is defined by the μ and is not shadowed by any pre-existing attribute. ∎

Proof of `thm:m2`. For each $r \in L$, LHS computes $e_1(r)$ and then $e_2(r)$, attaching both; RHS computes $e_2(r)$ and then $e_1(r)$. Because e_1 and e_2 depend only on r and not on each other's intermediate result, the two sequences yield the same extension $r \oplus \{c_1 \mapsto e_1(r), c_2 \mapsto e_2(r)\}$. The multisets are equal. ∎

Proof of `thm:m3`. For $r \in L$, the LHS produces $\pi_A(r) \oplus \{c \mapsto e(\pi_A(r))\}$. The RHS produces $r \oplus \{c \mapsto e(r)\}$ (using the same e since e depends only on A), then projects to $(A \cup \{c\})$. By the support of e, $e(r) = e(\pi_A(r))$, and the resulting tuples agree. The freshness of c ensures the projection is well-formed. ∎

Proof of `thm:m4`. Fix a row $(r_L, r_M) \in L \bowtie_\psi M$ such that $\psi(r_L, r_M)$ holds. The LHS produces $(r_L, r_M) \oplus \{c \mapsto e(r_L)\}$. The RHS first materializes L to obtain $\mu_{e \to c}(L) = \{r'_L \oplus \{c \mapsto e(r'_L)\} : r'_L \in L\}$, then joins with M on ψ; since c is fresh and not referenced by ψ (Def. fresh), the result is $\{(r'_L, r_M) \oplus \{c \mapsto e(r'_L)\} : r'_L \in L, r_M \in M, \psi(r'_L, r_M)\}$. Because e depends only on L's attributes, the materialized value $e(r_L)$ is computed in both cases from the same r_L, and ψ is unaffected by the addition of c to L's schema. The two multisets agree row-by-row. ∎

Proof of `unlabeled`. For each row $r \in L$, the LHS first expands r via $\Gamma_{g,d}$ to obtain a multiset $\{(r, v_i) : v_i$ reachable from r.id by g within $d\}$ (multiplicity = number of paths), then materializes e on each traversal row. The result is the multiset $\{(r, v_i, c) : r \in L, v_i$ reachable from $r, c = e(r)\}$, where e depends only on r's attributes by the support condition. The RHS first materializes e on L to obtain $\mu_{e \to c}(L) = \{r \oplus \{c \mapsto e(r)\} : r \in L\}$, then expands via $\Gamma_{g,d}$. Because c is fresh and is not the graph rule's node-id attribute, the expansion proceeds from the original node-id (unchanged by the μ); the result is $\{((r, c), v_i) : r \in L, v_i$ reachable from r.id$\}$. Reordering attributes, this is $\{(r, v_i, c) : r \in L, v_i$ reachable from $r, c = e(r)\}$, the same multiset as the LHS. The two multisets agree row-by-row. ∎

Proof of `thm:g1`. The graph expansion $\Gamma_{g,d}$ is *row-multiplicating*: each origin row $r \in L$ produces one or more traversal rows in the result, and the multiplicity of r's descendants is preserved (i.e. $\Gamma_{g,d}$ does not collapse duplicates introduced by L). Let $R \subseteq L$ be the multiset of rows satisfying φ. The LHS starts from L and applies σ_φ *after* expansion, so any traversal row whose origin row lies outside R is removed; the RHS starts from R and only generates traversal rows whose origin satisfies φ, so none are removed. The multisets agree row-by-row, because φ references only origin attributes, if φ referenced traversal attributes, the two sides would diverge and the identity would fail. ∎

Proof of `unlabeled`. For each $r \in L$, the LHS expands r by g_2 to depth d_2 yielding rows (r, v_2), then expands each (r, v_2) by g_1 to depth d_1 yielding (r, v_2, v_1) for each v_1 reachable from v_2. The RHS expands r by $g_1; g_2$ to depth $d_1 + d_2$, yielding (r, v_1) for each v_1 reachable from r by the composite. By the disjoint-label condition, g_1's traversal starts at v_2 and ends at v_1, independent of r's attributes; collapsing intermediate v_2 in the LHS gives the same (r, v_1) multiset as the RHS. The non-overlap condition (g_1's traversal result disjoint from g_2's source) ensures no join is needed; without it, the LHS would require a join between g_2's targets and g_1's origins. ∎

Proof of `thm:rbo-confluence`. The ARS $\to_{\text{RBO}}$ is the union of the seven foundational rewrites (M1–M5, G1, G2) with the classical A1–A8 and the derived / implied rules above, oriented toward the optimized form. **Termination**: M1, M3, M4, G1 strictly decrease a weight function w that counts, for a plan P, the number of σs on μ-bound attributes, the number of redundant μs, and the number of πs that drop needed attributes. M2, M5, **G2**, and the implied σ-τ rewrite are zero-weight: M2 is a μ-reorder bounded by the $k!$ orderings of k materializations; M5 is a μ-Γ reorder bounded by the number of μ-Γ inversions; **G2 is a** Γ-Γ **collapse** bounded by the number of nested-Γ inversions (one collapse per nesting); the σ-τ rewrite is bounded by the number of σ-τ inversions. The topK recognition $(\tau_O \Lambda_K \leftrightarrow \text{topK}_O$ per the definition $\text{topK}_O(L, K) := \tau_O(\Lambda_K(L)))$ is a definitional rewrite applied by the CBO as a physical-realization choice (in one-to-one correspondence with its inverse, so it trivially terminates). Once w reaches a local minimum, only the zero-weight rewrites remain, and each terminates by its bounded inversion count. **Local confluence**: the critical pairs from overlapping redexes (M1∩M1, M1∩M4, M1∩G1, M1∩M5, M4∩G1, M4∩M5, M5∩G1, G1∩A4, **G2∩G2, G2∩G1, G2∩M5**, M2∩M1, M2∩M5, σ-τ ∩ M1, σ-τ ∩ M3) all join by further rewrites, giving local confluence. The G2∩G2 pair joins by a single G2 collapse (the outermost redex fires first; the resulting $\Gamma_{g_1;g_2,d_1+d_2}$ has no remaining Γ-Γ redex). G2∩G1 joins by G1 then G2 (σ pushes through the inner Γ, then the outer collapse). G2∩M5 joins by M5 then G2 (μ moves past the inner Γ first, then collapse). (A1-vector and A4-vector, V1, J1 in the codebase, are A1/A4 applied to vector predicates and do not produce new critical pairs beyond those of A1, A4.) By Newman's lemma, termination plus local confluence implies confluence, and the normal form is unique. ∎

C Counterexamples for N1–N5

This appendix collects the counterexamples and remarks referenced by Theorems 9–13 in Section 4.6.

N1 (HNSW approximation). Let R consist of a single row $r = (id : 1, \textbf{\textit{emb}} : \boldsymbol{v})$ with $\|\boldsymbol{v} - \boldsymbol{q}\| = 0.99\tau$ (just under the threshold). An HNSW index on ***emb*** configured with a small `ef` and an unfavourable entry point may fail to reach r during the graph walk, returning an empty result. A linear scan over R returns $\{r\}$. Both are valid physical implementations of the same logical selection. *Consequence:* the optimizer models approximation as a first-class cost dimension with quantified recall/precision (Section 7).

N2 (Similarity join asymmetry). Let $|R| = 10^6$ with an HNSW index on V_a, and $|S| = 10^3$ with no index. "HNSW on R probed by each row of S" issues 10^3 probes; the reverse would require either building an HNSW index on S or issuing 10^6 probes. The two logical plans are equivalent; the physical plans differ by three orders of magnitude. *Consequence:* the cost model encodes which side has the index; the RBO generates both LHS and RHS orderings, and the CBO picks the cheaper.

N3 (Limit and order do not distribute over union). Let R and S each contain 100 rows with a single scoring column; draw $R.\text{score} \sim \mathcal{U}([0,1])$ and $S.\text{score} \sim \mathcal{U}([9,10])$. The LHS, $\Lambda_5\ \tau_O(R \cup S)$, returns the 5 globally smallest scores (all from R, in $[0, 0.05]$); the RHS, $\Lambda_5\ \tau_O(R) \cup \Lambda_5\ \tau_O(S)$, returns 10 rows spanning the disjoint score ranges. *Consequence:* the optimizer must compute the union before applying the limit; the distributed-execution story is the CBO's, not the RBO's.

N4 (μ through Γ on the traversal column). Let L have rows $(id, \boldsymbol{emb})$, and let $\Gamma_{\text{follows},1}$ produce an additional $\boldsymbol{followed_emb}$ attribute. Take $e := \|\boldsymbol{followed_emb} - \boldsymbol{q}\|$, a distance to the *followed* node's embedding. The LHS $\mu_{e\to c}\ \Gamma(L)$ materializes c against the followed-node vector; the RHS $\Gamma_{\text{follows},1}\ \mu_{e\to c}(L)$ first materializes $c' = \|\boldsymbol{emb} - \boldsymbol{q}\|$ against the origin vector and then expands, the two multisets differ row-by-row. *Consequence:* the RBO's M5 match verifies (via a $\mu \to \Gamma$ compatibility check) that the materialization expression references only origin attributes.

N5 (Λ/τ do not commute with Γ). Let L have 1000 origin rows with a *score* column, and let $\Gamma_{\text{cites},1}$ expand each origin to a variable $1 \le n_i \le 50$ citing nodes. The LHS $\Lambda_{10}\ \tau_{score}(\Gamma(L))$ returns the 10 expansion rows with the globally smallest *score*; the RHS $\Gamma(\Lambda_{10}\ \tau_{score}(L))$ first picks the 10 origin rows with the smallest *score* and then expands, returning up to $10 \cdot 50 = 500$ rows. *Consequence:* the RBO never applies the Λ/τ pushdown across Γ; the choice is the CBO's via the cost model.

D SQL Desugaring Details

This appendix collects the SQL desugaring rules referenced informatively in Section 5.

D1. Projection + filter + order + limit. For a query of the form

```
SELECT expr AS c,. FROM L WHERE phi ORDER BY O LIMIT K
```

the desugaring is $\Lambda_K(\tau_O(\sigma_\varphi(\mu_{e\to c}(L))))$ with μ chaining for multiple aliased expressions.

D2. Vector predicate. $\mathcal{D}\langle$ emb IS NEAR:q WITHIN DISTANCE:t $\rangle_{\text{pred}} = \sigma_{\|emb-q\|<t}$. The bound parameter :q and threshold :t are the query-time bound parameters of Section 3; the desugared form is a first-class algebra expression, so the optimizer's selection-pushdown rules (J1, M1) apply.

D3. Text predicate. $\mathcal{D}\langle$ text MATCHES:q WITH SCORE >:$\theta\rangle_{\text{pred}} = \sigma_{\text{BM25}(text,q)>\theta}$. The (k_1, b) parameters are carried with the BM25 index and looked up at planning time; the CBO's inverted-index bridge (Section 7) applies directly to the desugared form.

Algorithm 4 RBO fixed-point loop (A, Section ??).

Input: logical plan P; matcher list $\mathcal{M}$ = (M1, M3, M4, G1, G2, M5, M2, V1, J1, T1); `max_iters` safety bound (default 64). **Output:** rewritten plan P'; trace τ of applied rules.

```
τ ← []
repeat
    changed ← FALSE
    for each node n in P in preorder do
        for each m_i ∈ M in priority order do
            if m_i.MATCH(n) returns rewrite n' then
                n ← n'; τ.APPEND(m_i.NAME); changed ← TRUE; break
            end if
        end for
    end for
until ¬changed or |τ| = max_iters
return P, τ
```

D4. Graph predicate. $\mathcal{D}\langle$ id IS REACHABLE FROM:n WITHIN 2 HOPS VIA:g $\rangle_{\text{p}}$ $\sigma_{id\in\pi_{dst}(\Gamma_{g,d=2}(\sigma_{id=n}(\cdot)))}$.

D5. Materialization alias. $\mathcal{D}\langle$ expr AS c $\rangle = \mu_{e\to c}$ where e is the desugared expression and c is the alias.

Worked example ($Q_1 \to Q_6$). The full SQL surface for the $Q_1 \to Q_6$ running example:

```
WITH scored AS MATERIALIZED (
    SELECT id, text, ts, subsystem,
           emb <=>:q1     AS d1,
           emb_alt <=>:q2 AS d2
    FROM memory
    WHERE subsystem = 'fault-tolerance'
      AND ts > now() - interval '30 days'
)
SELECT id, text, ts
FROM scored
WHERE d1 <:tau1 AND d2 <:tau2
ORDER BY 0.7 * d1 + 0.3 * d2
LIMIT 10;
```

desugars as follows: the `WITH scored AS MATERIALIZED` sub-query becomes $\mu_{d_1,d_2}(\sigma_{\text{subsystem}\wedge\text{ts}}(Mem))$; the outer `SELECT` to $\pi_{id,text,ts}$; the `WHERE` to $\sigma_{d_1<\tau_1\wedge d_2<\tau_2}$; and the `ORDER BY` d_1, d_2 `... LIMIT 10` to Λ_{10} per T1 (an explicit score-fusion policy such as `ORDER BY` $0.7\,d_1 + 0.3\,d_2$ is also expressible via the τ operator and desugars to $\tau_{0.7\,d_1+0.3\,d_2}\ \Lambda_{10}$). The optimizer then applies M1: the predicate $d_1 < \tau_1$ is pushed below the materialization that binds d_1, so $\|\boldsymbol{emb} - \boldsymbol{q}_1\|$ is computed only on rows that pass the subsystem filter. The user wrote the SQL in the order that reads naturally; the optimizer delivers a plan that computes d_1, d_2 only on survivors.

E Algorithm Pseudocode

This appendix collects the four algorithm boxes referenced from Sections 6–??.

Per-strategy cost table (Section ??).

Algorithm 5 Approximation-aware CBO search (A).

Input: logical plan P; budget $\rho_{\min}$; cost model C. **Output:** optimal $P^\star$ (or Pareto frontier).

```
frontier ← ∅
S ← PhysicalRealizations(P)
for each P_phys ∈ S do
    c ← C(P_phys)
    if c.approx > ρ_min then
        continue
    end if
    if DominatedBy(P_phys, frontier) then
        continue
    end if
    AddToFrontier(P_phys, frontier)
    PruneDominated(frontier)
end for
return SelectFromFrontier(frontier)
```

Algorithm 6 Adaptive re-planning at a plan point (B).

Input: current physical plan P; budget $\rho_{\min}$; candidate set C from the prefix of P. **Output:** continue, or replace suffix of P with higher-recall realization.

```
r ← SampledRecall(C, sample_size)
if r ≥ ρ_min then
    return Continue(C)
end if
P' ← HigherRecallRealization(P)
return ReplaceSuffix(P, P')
```

Algorithm 7 Higher-recall realization precomputation (called by Algorithm 3).

Input: current physical realization P; budget $\rho_{\min}$; cost model C. **Output:** ordered list $\mathcal{H}(P)$ of higher-recall candidates.

```
H ← ∅
for each modality m that P realises do
    for each higher-recall parameter θ' of m do
        P' ← P.WithParameter(m, θ')
        if C(P').approx > C(P).approx then
            InsertSorted(P', H)
        end if
    end for
end for
if H = ∅ and P contains HNSW then
    InsertSorted(P' (with m replaced by flat), H)
end if
return H
```

The HNSW cost is amplified by $1/\rho$ when the surviving subset fragments the HNSW graph; recall is multiplied by $(0.5 + 0.5\rho)$. The CBO enumerates the three strategies, picks the lowest-time strategy meeting the $\rho_{\min} = 0.95$ floor; HNSW is skipped when $\rho < 0.3$.

F Discussion

F.1 Limitations

(i) **Single-node, single-user:** no partitioning / replication / isolation; cost model has no cross-node I/O term. (ii) **Read-only:** no write path; algebra is defined on a snapshot. (iii) **Approximation is a cost, not a contract:** the cost model treats recall as a dimension; the algebra treats queries as exact. (iv) **Cost-model calibration** is empirical and not yet performed; a poorly calibrated cost model is worse than none.

F.2 Open problems

(i) **Write path** (log-structured with deferred index updates); (ii) **Multi-tenancy** (`tenant` as a first-class attribute); (iii) **Distributed execution** (N3 blocks naive topK distribution; a partition-aware topK algorithm that respects N3 is a natural direction); (iv) **Learned indexes** [19] (a drop-in replacement for a physical selection operator).

F.3 Connection to other research

Three threads: *linear algebra* (vector distance is $\|V - q\|$ and an HNSW probe is a linear-algebra computation, fusing linear-algebra kernels with relational operators at the physical level is the natural direction); *probabilistic databases* [5] (HNSW approximation is a probability distribution; the cost model's approximation dimension is a step toward treating the database as a probabilistic system); *programming-language theory* (μ is, in spirit, a `let` binding with a hygienic freshness condition; a type-theoretic treatment would strengthen the foundation).

F.4 Closing

The argument of this paper is simple: agentic memory is multi-modal, multi-modal is a query problem, query problems benefit from foundations, and the right foundation is an extension of relational algebra. We claim not to have solved agentic memory, but that the field will benefit from a shared vocabulary, and we offer MosaicOpt as a starting point for that conversation.

Strategy	time (ms)	memory (MB)	recall	rec (ms)
`flat`	$0.04 + 0.06\, n_i d/10^6$	$4n_i d/2^{20}$	1.0 (exact)	0
`ivf`	$0.5 + 0.01\, n_i d/10^6$	$0.20 \cdot 4n_i d/2^{20}$	$0.98 - 0.02e^{-30s}$	$5 + 0.0005 n_i$
`hnsw`	$(0.1 + 0.0008\, n_i d/10^6)/\rho$	$1.30 \cdot 4n_i d/2^{20}$	$(1 - 0.04e^{-5s}) \cdot (0.5 + 0.5\rho)$	$50 + 0.001 n_i$